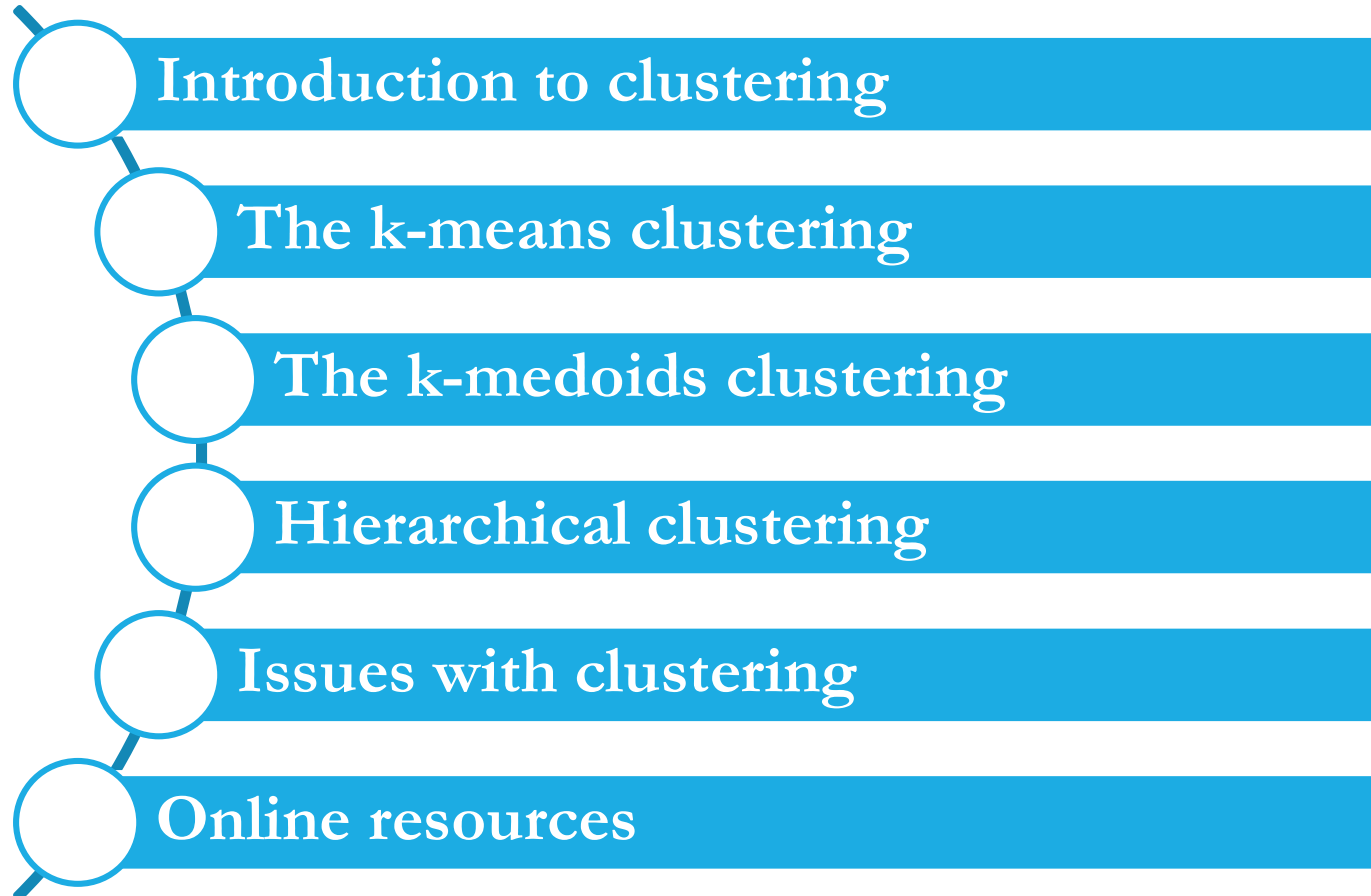


Clustering



Introduction to clustering

Clustering is the process of organizing objects into groups whose members are similar in some way. Whereas, customer segmentation is the practice of dividing a customer base into groups of individuals that are similar in specific ways relevant to marketing, such as age, gender, interests, spending habits and so on.

Customer segmentation or clustering is useful in many ways. It could be used for targeted marketing. Sometimes when building predictive model, it is rather effective to cluster the data and build a separate predictive model for each cluster.

Clustering is an undirected data mining technique. This means it can be used to identify hidden patterns and structures in the data without formulating a specific hypothesis. There is no target variable in clustering. For example, the grocery retailer was not actively trying to identify fresh food lovers at the start of the analysis. It was just attempting to understand the different buying behaviours of its customer base.



Introduction to clustering

Clustering is performed to identify similarities with respect to precise behaviours or dimensions. For instance, we want to identify customer segments with similar buying behaviour. Hence, clustering was performed using variables that represent the customer buying patterns.

Cluster analysis can be used to discover structures in data without providing an explanation or interpretation. Cluster analysis simply discovers patterns in data without explaining why they exist. The resulting clusters are meaningless by themselves. They need to be profiled extensively to build their identity, that is, to understand what they represent and how they are different from the parent population.

Clustering is primarily used to perform segmentation, be it customer, product or store. For example, products can be clustered together into hierarchical groups based on their attributes like use, size, brand, flavour, etc.; stores with similar characteristics – sales, size, customer base, etc. – can be clustered together.



Introduction to clustering

Clustering procedure can be hierarchical where clustering is characterized by the development of a hierarchy or treelike structure.

- Agglomerative clustering starts with each object in a separate cluster and clusters are formed by grouping objects into bigger and bigger clusters.
- Divisive clustering on the other hand starts with all the objects grouped into a single cluster and clusters are then divided or split until each object is in a separate cluster.
- K-means clustering is a non-hierarchical clustering and is a procedure which first assigns or determines a cluster centre and then groups all the objects within a pre-specified threshold value together working out from the centre.

Deciding on the number of clusters is based on theoretical or practical considerations. In hierarchical clustering the distances at which clusters are combined can be used as criteria. In non-hierarchical clustering the ratio of the total within group variance to between group variance can be plotted against the number of clusters.



Introduction to clustering

Interpreting and profiling the clusters involves examining the cluster centroids.

The centroids represent the mean values of the objects contained in the cluster on each of the variables. The centroids can be assigned with a name or label.

To assess reliability and validity one has to perform cluster analysis on the same data using different distance measures and compare the results to determine stability of solutions.

Splitting the data randomly into halves and performing clustering separately on each half and comparing cluster centroids across two sub-samples is one way to do it.

In hierarchical clustering, the solution may depend on the order of cases in the data set. To achieve the best results, make multiple runs using different order of cases until the solution stabilizes.



Introduction to clustering

Clustering can also be used for anomaly detection, for example, identifying fraud transactions.

Cluster detection methods can be used on a sample containing only valid transactions to determine the shape and size of the “normal” cluster. When a transaction comes along that falls outside the cluster for any reason, it is suspect.

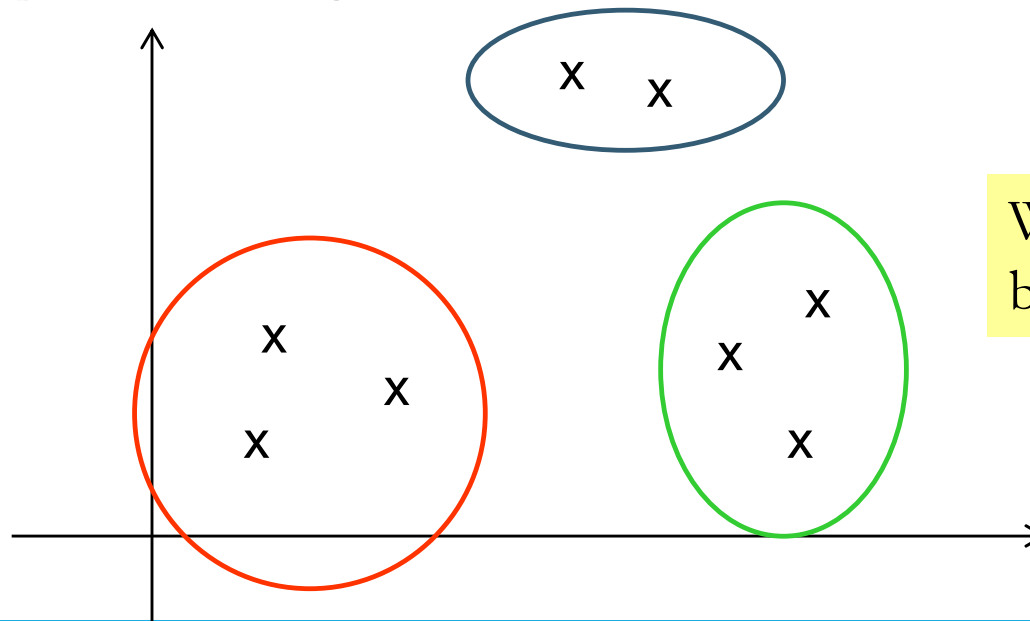
This approach has been used in medicine to detect the presence of abnormal cells in tissue samples and in telecommunications to detect calling patterns indicative of fraud.

Clustering is often used to break large set of data into smaller groups that are more amenable to other techniques. For example, logistic regression results can be improved by performing it separately on smaller clusters that behave differently and may follow slightly different distributions.



Introduction to clustering

- The goal of clustering is to
 - group data points that are close (or **similar**) to each other
 - identify such groupings (or clusters) in an **unsupervised** manner
- Why unsupervised? There is no information provided to the algorithm on which data points belong to which clusters

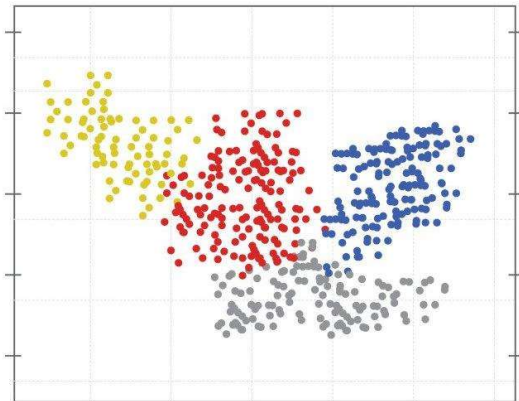


What should the clusters be for these data points?



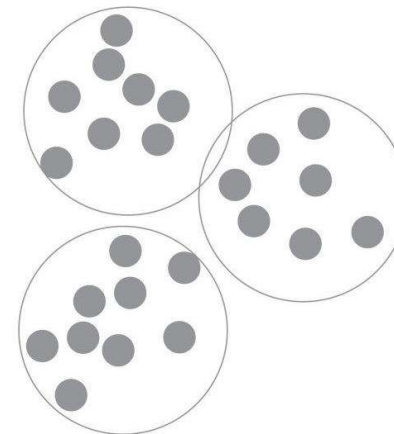
Cluster analysis

Grouping **entities** is based on distance (proximity)



Factor analysis

Grouping **variables** is based on patterns of variation (correlation)



Data structures

***Data* matrix**

$$\begin{array}{c} \text{attributes/dimensions} \\ \left. \begin{array}{c} \text{objects} \end{array} \right\} \begin{bmatrix} x_{11} & \dots & x_{1\ell} & \dots & x_{1d} \\ \dots & \dots & \dots & \dots & \dots \\ x_{i1} & \dots & x_{i\ell} & \dots & x_{id} \\ \dots & \dots & \dots & \dots & \dots \\ x_{n1} & \dots & x_{n\ell} & \dots & x_{nd} \end{bmatrix} \end{array}$$

***Distance* matrix**

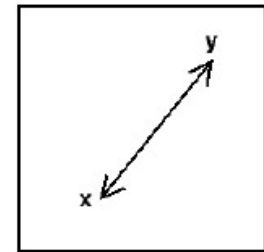
$$\begin{array}{c} \text{objects} \\ \left. \begin{array}{c} \text{objects} \end{array} \right\} \begin{bmatrix} 0 & & & & \\ d(2,1) & 0 & & & \\ d(3,1) & d(3,2) & 0 & & \\ \vdots & \vdots & \vdots & & \\ d(n,1) & d(n,2) & \dots & \dots & 0 \end{bmatrix} \end{array}$$



Distance measures

- Euclidean distance

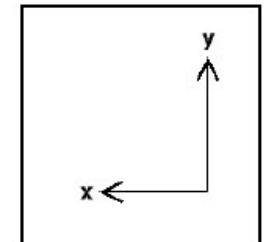
$$d(g_1, g_2) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$



Euclidean

- Manhattan distance

$$d(g_1, g_2) = \sum_{i=1}^n |(x_i - y_i)|$$



Manhattan

- Minkowski distance

$$d(g_1, g_2) = \sqrt[m]{\sum_{i=1}^n (x_i - y_i)^m}$$



Distance measures

- **Euclidean distance** is the most common choice of similarity measure for microarray data
- If variables are measured in vastly different units, we must **standardize data**
- Also **outliers** should be inspected
- Using **different measures** for clustering can yield **different clusters**
- It is advisable to use different measures and **compare** the results.



- Cluster analysis is used for:
 - Data reduction
 - Hypothesis generation
 - Classification of novel entities
- Cluster analysis is criticized for:
 - Being descriptive, atheoretical, and noninferential
 - Always creating clusters, no matter if they actually exist or no
 - Its solution is not generalizable



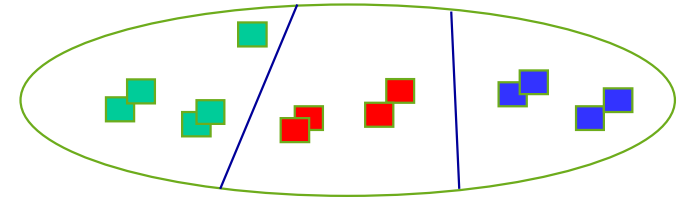
A Categorization of major clustering methods

- Partitioning methods
- Hierarchical methods
- Density-based methods
- Grid-based methods



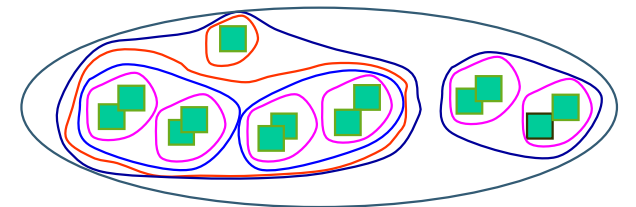
Major clustering methods

Partitioning algorithms: Partition a dataset to k clusters, e.g., $k=3$

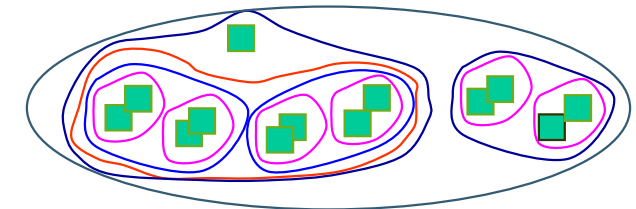


Hierarchical algorithms: Create hierarchical decomposition of ever-finer partitions.

top down (divisive)

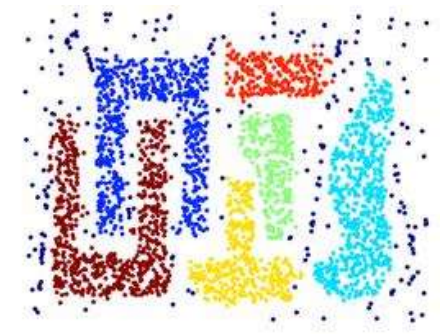


bottom up (agglomerative)

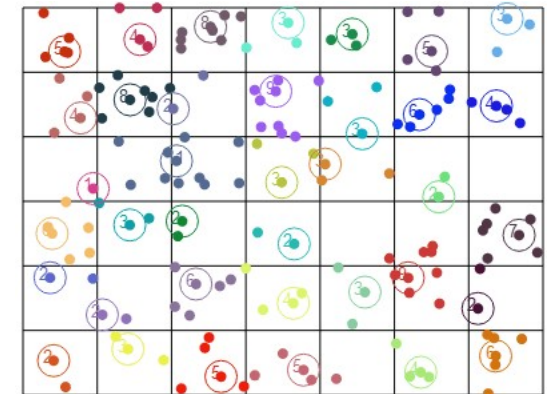


Major clustering methods

Density-based algorithms: Discover clusters of arbitrary shape



Grid-based algorithms: Divide the space into infinite number of cells and calculate the cell density



Clustering Methods based on Partitioning

- Partitioning method: Construct a partition of a database D of n objects into a set of k clusters
- Given a k , find a partition of k clusters that optimizes the chosen **partitioning criterion**
 - **k-means** (MacQueen, 1967): Each cluster is represented by the **center** of the cluster
 - **k-medoids** or PAM method (Partition Around Medoids) (Kaufman & Rousseeuw, 1987): Each cluster is represented by **1 object** in the cluster



The *K-Means* clustering method

Given k , the *k-means* algorithm is implemented in 4 steps:

1. Partition objects into k nonempty subsets (or pick k initial means).
2. Compute the *mean (center) or centroid* of each cluster of the current partition (if one started with k means, then this step is done).
centroid $\sim$ point that minimizes the *sum of dissimilarities from the mean or the sum of the square errors from the mean.*
Assign each object to the cluster with the most similar (closest) center.
3. Go back to Step 2
4. Stop when the new set of means doesn't change (or some other stopping condition?)



The *K-Means* clustering method

- **Properties of k-means**
 - Guaranteed to converge
 - Guaranteed to achieve local optimum, not necessarily global optimum
- **Advantages**
 - Low complexity
 - Easy to interpret
- **Disadvantages**
 - Necessity of specifying the number of clusters
 - Sensitive to noise and outlier data points
 - Clusters are sensitive to initial assignment of centroids



K-means clustering in R

```
install.packages("foreign")
```

```
library(foreign)
```

```
dataset <- read.spss(file.choose(),  
to.data.frame=TRUE)
```

```
data.use <- subset(dataset, select=-  
c(University))
```

- Install package foreign
- Call the package foreign
- Load the data set (bricsfull.sav)
- Remove the University name from the dataset



K-means clustering in R

```
install.packages("cluster")
```

```
library(cluster)
```

```
kmeanscluster<-kmeans(data.use, 4)
```

```
table(kmeanscluster$cluster)
```

- Install package cluster
- Call the package cluster
- Perform the k-means clustering and retain 4 clusters
- List out the number of entities in each cluster



K-means clustering in R

```
install.packages("cluster")
```

```
library(cluster)
```

```
kmeanscluster<-kmeans(data.use, 4)
```

```
table(kmeanscluster$cluster)
```

- Install package cluster
- Call the package cluster
- Perform the k-means clustering and retain 4 clusters
- List out the number of entities in each cluster

Cluster	1	2	3	4
Number of entities	14	8	18	10



K-means clustering in R

```
install.packages("cluster")
```

```
library(cluster)
```

```
kmeanscluster<-kmeans(data.use, 4)
```

```
table(kmeanscluster$cluster)
```

```
kmeanscluster$centers
```

- Install package cluster
- Call the package cluster
- Perform the k-means clustering and retain 4 clusters
- List out the number of entities in each cluster
- List out each cluster centre



K-means clustering in R

```
install.packages("cluster")
```

```
library(cluster)
```

```
kmeanscluster<-kmeans(data.use, 4)
```

```
table(kmeanscluster$cluster)
```

```
kmeanscluster$centers
```

- Install package cluster
- Call the package cluster
- Perform the k-means clustering and retain 4 clusters
- List out the number of entities in each cluster
- List out each cluster centre



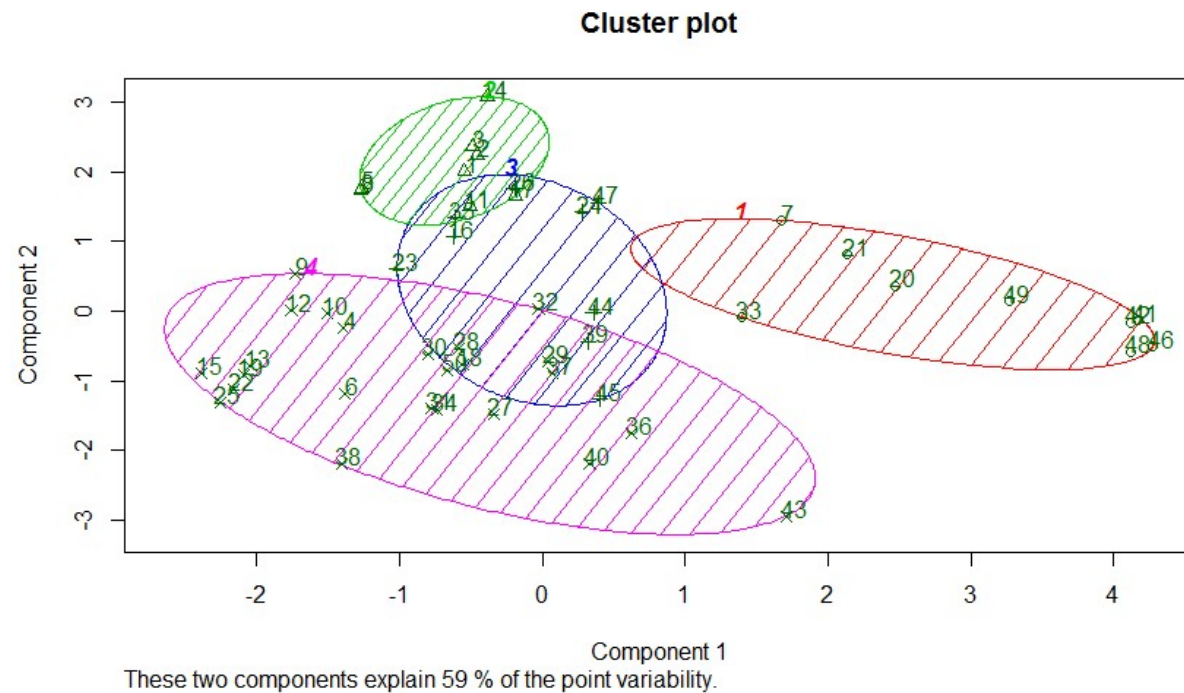
K-means clustering in R

	Academic Reputation	Employer Reputation	Faculty Student	Staff with PhD	Papers per Faculty	Citations per paper	International faculty	International Students
1	95,486	92,636	52,586	79,986	87,329	88,700	98,300	72,214
2	76,388	80,175	97,850	83,525	23,663	19,150	47,363	87,925
3	88,589	89,017	49,589	95,628	96,067	93,778	24,856	17,439
4	87,850	74,700	47,650	90,420	55,590	55,030	52,680	17,730



K-means - Plot

```
install.packages("fpc")  
library(fpc)  
clusplot(data.use, kmeanscluster$cluster, color=TRUE, shade=TRUE, labels=2,  
lines=0, main="Cluster plot")
```



The *K-Medoids* clustering method

- Find *representative* objects, called medoids, which must be objects in the cluster
- Given k , *PAM* (Partitioning Around Medoids) algorithm is implemented in four steps:the
 1. Start from an initial set of medoids
 2. Iteratively replace one of the medoids by a non-medoid
 3. Go back to step 2. Do this over all medoid-nonmedoid pairs
 4. Stop if the aggregate similarity measure is improved.
- *PAM* works for small data sets. Does not scale for large data sets



K-medoids – pam(x)

```
data.use = subset(dataset, select=-  
c(University))
```

```
pamcluster<-pam(data.use, 4, metric =  
"euclidean")
```

```
summary(pamcluster)
```



```
table(pamcluster$clustering)
```

- Remove the University name from the dataset
- Perform the pam clustering and retain 4 clusters
- Provide with the basic info on the clusters – as it lists many parameters it is better to demand particular parameters
- List out the number of entities in each cluster



K-medoids – pam(x)

```
data.use = subset(dataset, select=-  
c(University))
```

```
pamcluster<-pam(data.use, 4, metric =  
"euclidean")
```

```
summary(pamcluster)
```



```
table(pamcluster$clustering)
```

- Remove the University name from the dataset
- Perform the pam clustering and retain 4 clusters
- Provide with the basic info on the clusters – as it lists many parameters it is better to demand particular parameters
- List out the number of entities in each cluster

Cluster	1	2	3	4
Number of entities	14	19	8	9



K-medoids – pam(x)

```
data.use = subset(dataset,select=-  
c(University))
```

```
pamcluster<-pam(data.use, 4, metric =  
"euclidean")
```

```
summary(pamcluster)
```



```
table(pamcluster$clustering)
```

```
pamcluster$id.med
```

- Remove the University name from the dataset
- Perform the pam clustering and retain 4 clusters
- Provide with the basic info on the clusters – as it lists many parameters it is better to demand particular parameters
- List out the number of entities in each cluster
- List out the medoids



K-medoids – pam(x)

```
data.use = subset(dataset, select=-
c(University))

pamcluster<-pam(data.use, 4, metric =
"euclidean")

summary(pamcluster)

↓

table(pamcluster$clustering)

pamcluster$id.med
```

- Remove the University name from the dataset
- Perform the pam clustering and retain 4 clusters
- Provide with the basic info on the clusters – as it lists many parameters it is better to demand particular parameters
- List out the number of entities in each cluster
- List out the medoids

Cluster	1	2	3	4
Medoid	11	28	22	20



K-medoids – pam(x)

```
dataset$University[pamx$id.med]
```

- List out the medoids with their respective names



K-medoids – pam(x)

```
dataset$University[pamx$id.med]
```

- List out the medoids with their respective names

Cluster	1	2	3	4
Medoid	Beijing Normal University	Beijing Institute of Technology	Indian Institute of Technology Kanpur (IITK)	Saint-Petersburg State University



K-medoids – pam(x)

```
dataset$University[pamcluster$id.med]
```

```
dataset$University[pamcluster$clustering == 1]
```

- List out the medoids with their respective names
- List out universities by name which are in a particular cluster



K-medoids – pam(x)

```
dataset$University[pamcluster$id.med]
```

```
dataset$University[pamcluster$clustering == 1]
```

- List out the medoids with their respective names
- List out universities by name which are in a particular cluster

Universities in Cluster 1

Tsinghua University
Peking University
Fudan University
Shanghai Jiao Tong University
Nanjing University Beijing Normal
University
University of Cape Town

Wuhan University
Tongji University
Sun Yat-sen University
Xi'an Jiaotong University
University of The Witwatersrand
Stellenbosch University
University of Pretoria



K-medoids – pam(x)

```
dataset$University[pamcluster$id.med]
```

```
dataset$University[pamcluster$clustering == 1]
```

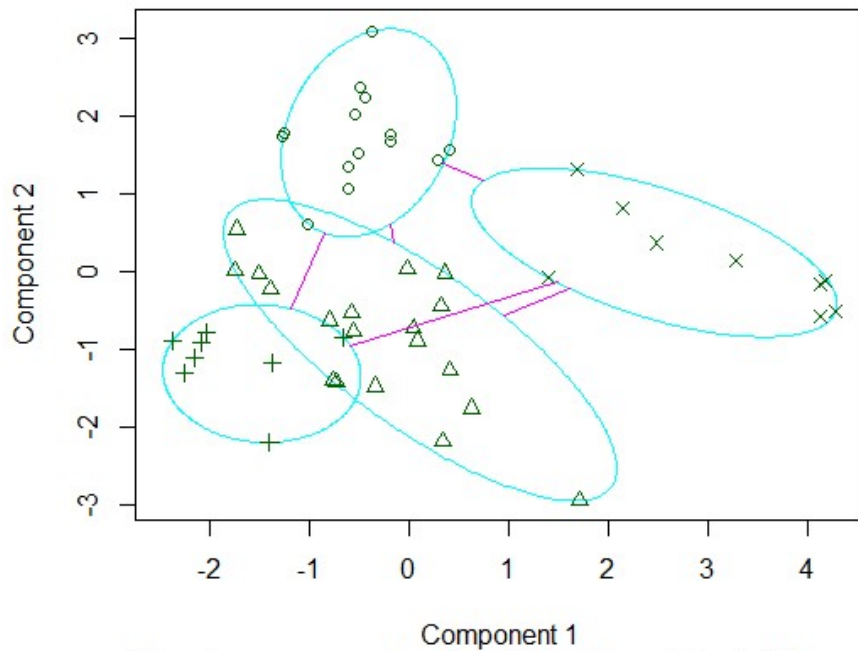
```
plot(pamcluster, main="Cluster plot of pam()")
```

- List out the medoids with their respective names
- List out universities by name which are in a particular cluster
- Gives the plot and the silhouette plot of the retained clusters (click back in the Plot section to view the Cluster plot)



K-medoids – pam(x)

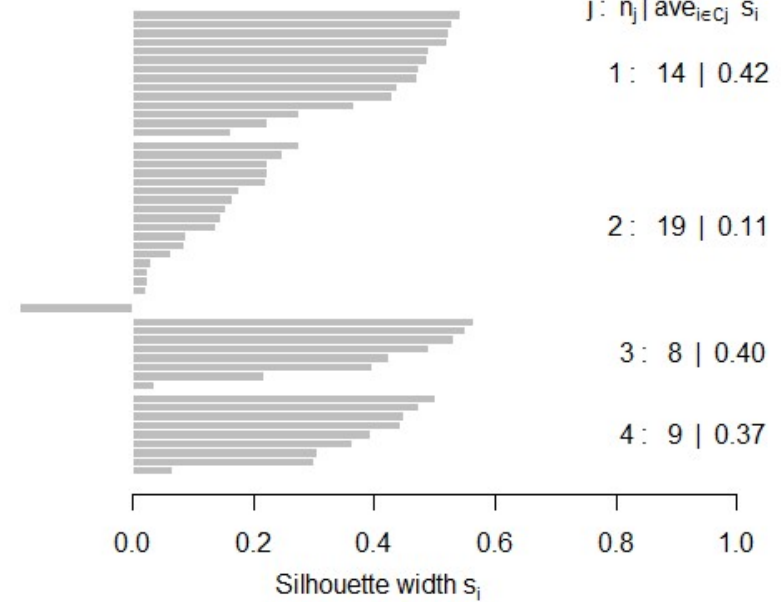
Cluster plot of pam()



These two components explain 59 % of the point variability.

Cluster plot of pam()

n = 50



Average silhouette width : 0.29



K-medoids – pamk(x)

- Function `pamk()` from the package `fpc` does not require the user to specify `k`

```
install.packages("fpc")
```

```
library(fpc)
```

```
pamk.result<-pamk(data.use)
```

```
pamk.result$nc
```

- Install the `fpc` package
- Load the `fpc` package
- Perform the clustering
- List out the number of clusters suggested by `pamk` algorithm



K-medoids – pamk(x)

- Function `pamk()` from the package `fpc` does not require the user to specify `k`

```
install.packages("fpc")
```

```
library(fpc)
```

```
pamk.result<-pamk(data.use)
```

```
pamk.result$nc
```

- Install the `fpc` package
- Load the `fpc` package
- Perform the clustering
- List out the number of clusters suggested by `pamk` algorithm

The retained number of clusters is 3



K-medoids – pamk(x)

```
table(pamk.result$pamobject$clustering,  
      pamcluster$clustering)
```

- Compare pam() and pamk() clusters

		Pam() clusters			
		1	2	3	4
Pamk(x) clusters	1	14	0	0	0
	2	0	19	8	0
	3	0	0	0	9



K-medoids – pamk(x)

```
table(pamk.result$pamobject$clustering,  
      pamcluster$clustering)
```

```
plot(pamk.result$pamobject, main="Cluster  
plot of pamk()")
```

- Compare pam() and pamk() clusters
- Gives the plot of the retained clusters

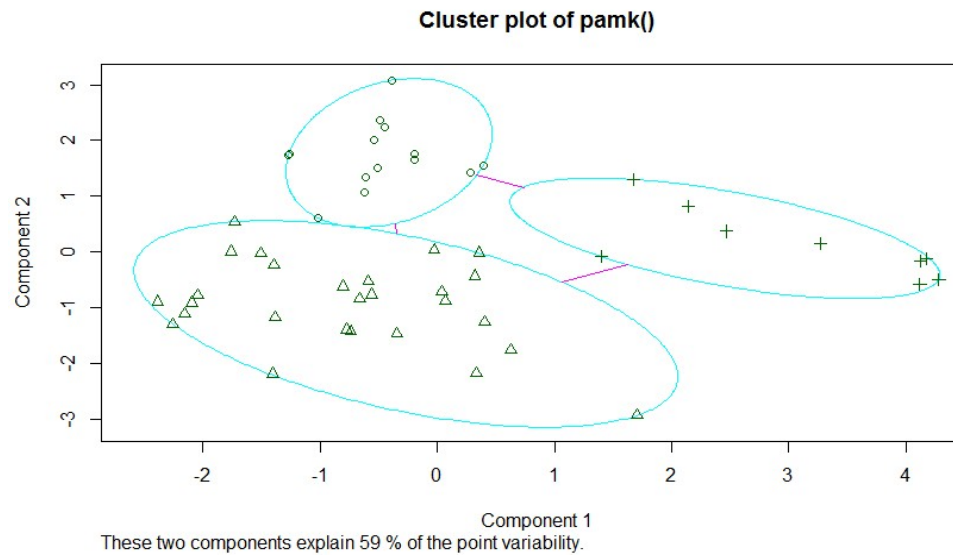


K-medoids – pamk(x)

```
table(pamk.result$pamobject$clustering,  
      pamcluster$clustering)
```

```
plot(pamk.result$pamobject, main="Cluster  
plot of pamk()")
```

- Compare pam() and pamk() clusters
- Gives the plot of the retained clusters



CLARA algorithm

- CLARA – enhanced PAM algorithm
- It draws multiple samples of data from the original sample, applying PAM to each sample and then returning the best clustering.
- It then assigns all objects in the dataset to these clusters.
- **Strength:** It performs better on larger data and has high speed
- **Weakness:** Efficiency depends on the sample size and it might not necessarily represent a good clustering of the whole data



As Clara works best with large data sets with more than 200 entities we need a new data file

We will observe the ARWU University Ranking for the year 2016 for all 500 universities

```
library(foreign)

arwu.data = read.spss("~Your folder
path~/ARWU 2016.sav", to.data.frame=TRUE)

arwu.use=subset(arwu.data, select=-c(Institution))
```

- Call the package foreign
- Load the data file
- Remove the Institution name from the dataset



```
clarax <- clara(x, k, samples=5, sampsize)
```

Function parameters of clara()

x – data matrix

k – number of clusters to be retained

samples – number of samples to be drawn – by default is 5

sampsize – size of the drawn samples – between k and n(row(x))

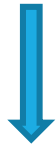


clara()

```
clarax <- clara(arwu.use, 5, samples=150,  
sampsiz=100)
```

clarax

- Call the clara() function
- Print the analysis output

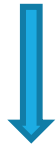


clara()

```
clarax <- clara(arwu.use, 5, samples=150,  
sampsiz=100)
```

clarax

- Call the clara() function
- Print the analysis output



Medoids and their ID

Medoids:

	Alumni	Award	HiCi	NampS	PUB	PCP
3	66.8	79.4	65.3	67.5	68.1	55.9
42	19.7	20.0	31.9	32.5	52.5	26.0
182	0.0	0.0	15.9	14.6	39.3	20.8
243	13.9	0.0	10.1	13.4	37.3	18.0
373	0.0	0.0	8.7	7.5	31.5	16.1

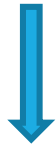


clara()

```
clarax <- clara(arwu.use, 5, samples=150,  
sampsiz=100)
```

clarax

- Call the clara() function
- Print the analysis output



Cluster sizes

```
cluster sizes:      11 82 112 98 197
```

Best sample

Best sample:

```
[1] 1 3 6 8 13 20 24 27 30 37 39 40 41 42 46 57 59 60 61 64 65 69 71 76  
[25] 79 80 81 82 102 105 115 124 128 133 140 145 147 148 153 155 168 175 182 186 191 196 198 223  
[49] 226 228 229 236 237 238 239 241 242 243 249 252 259 268 277 285 295 296 304 306 341 342 348 359  
[73] 368 371 372 373 383 388 392 394 401 402 407 415 416 426 429 434 435 437 438 447 458 459 462 474  
[97] 486 487 492 500
```



clara()

```
clarax <- clara(arwu.use, 5, samples=150,  
sampsiz=100)
```

```
clarax
```

```
arwu.data$Institution[clarax$cluster == 1]
```

```
arwu.data$Institution[clarax$i.med][1]
```

```
arwu.data$Institution[clarax$i.med]
```

- Call the clara() function
- Print the analysis output
- Print the universities which are in a particular cluster
- Print the university which is the centre of a particular cluster
- Print out the centres of all retained clusters



clara()

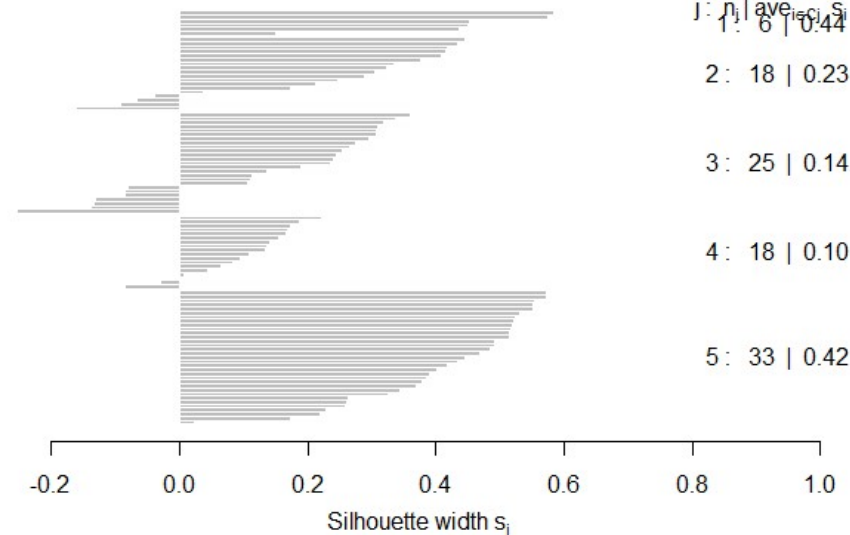
```
plot(clarax, main = "silhouette plot of clara() for 5 retained clusters")
```



Plots the Silhouette plot of the best sampled sample

Silhouette plot of clara() for 5 retained clusters

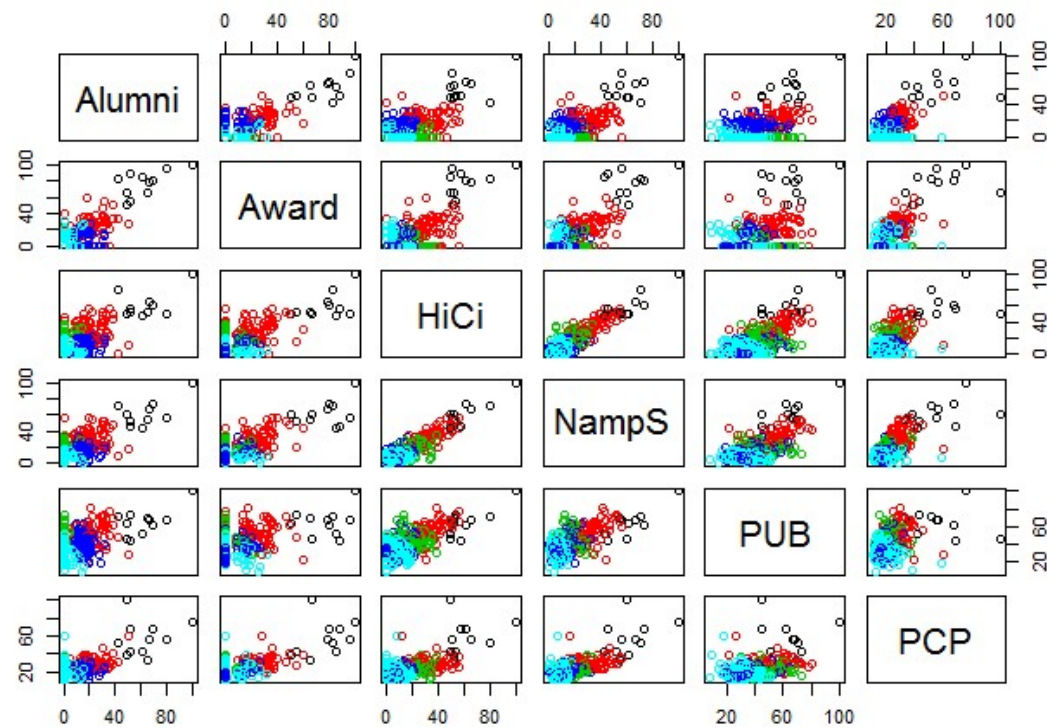
n = 100



```
plot(arwu.use, col = clarax$cluster)
```



Plots the pairwise variables by cluster

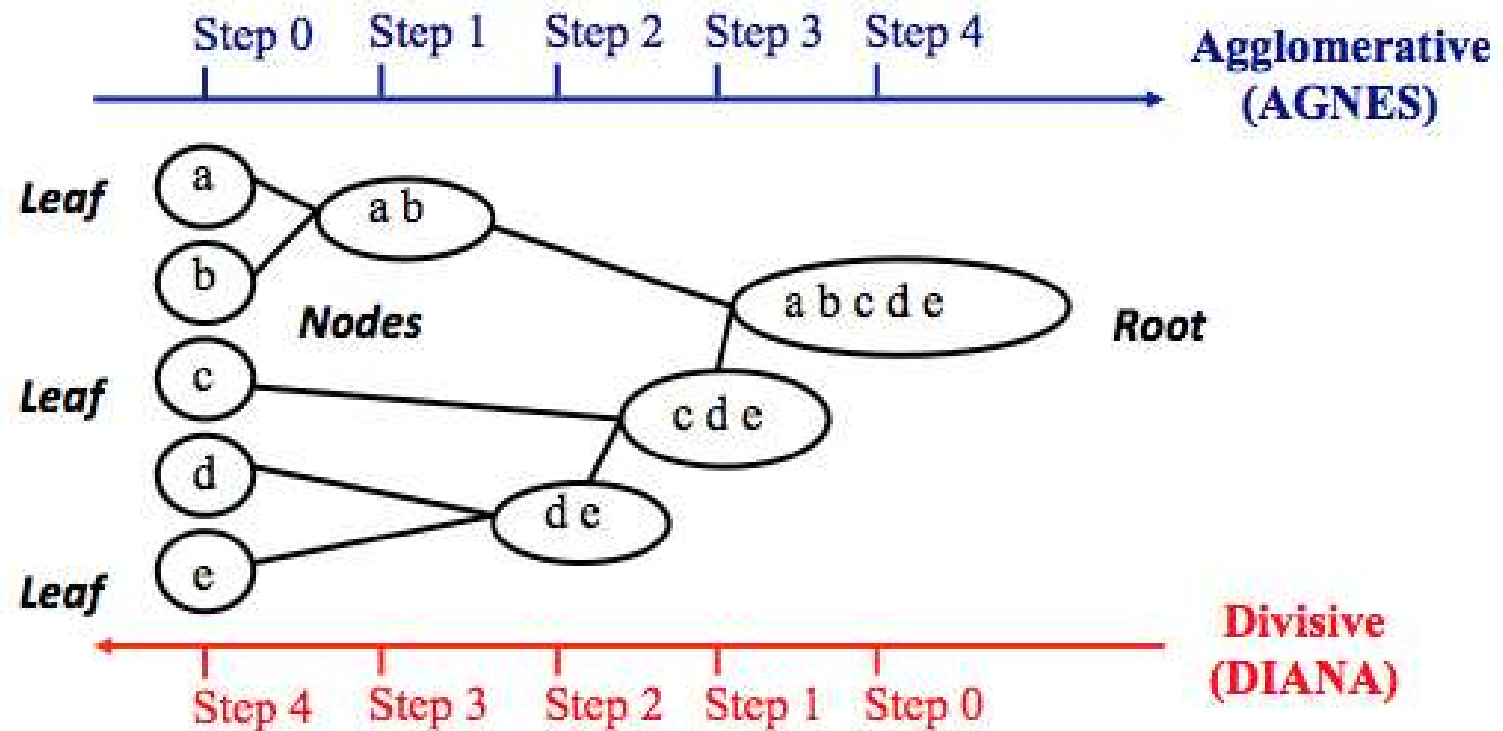


Hierarchical clustering

- There are two styles of hierarchical clustering algorithms to build a tree from the input set S :
 - **Agglomerative (bottom-up):**
 - Beginning with singletons (sets with 1 element)
 - Merging them until S is achieved as the root
 - It is the most common approach
 - **Divisive (top-down):**
 - Recursively partitioning S until singleton sets are reached

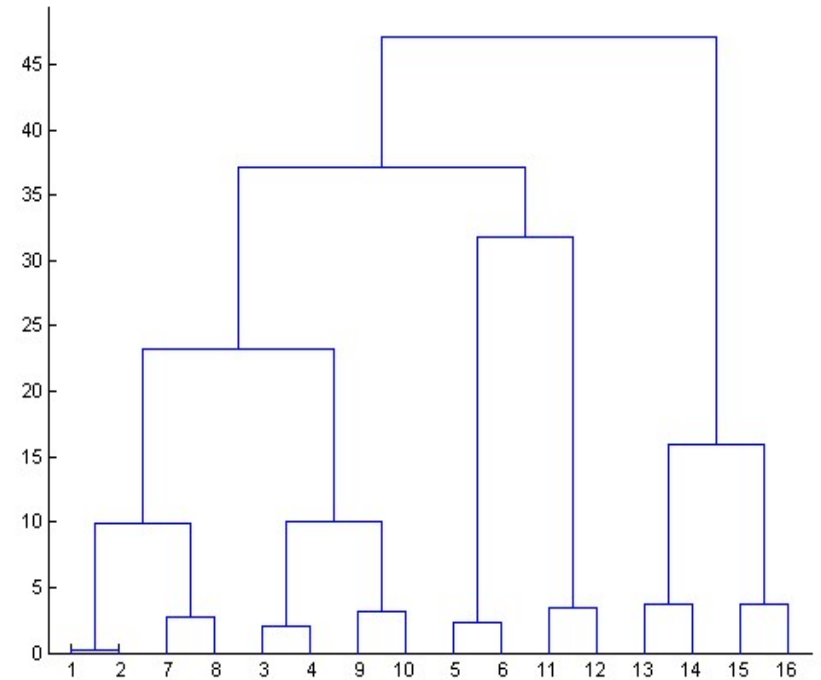


Hierarchical clustering



Hierarchical clustering

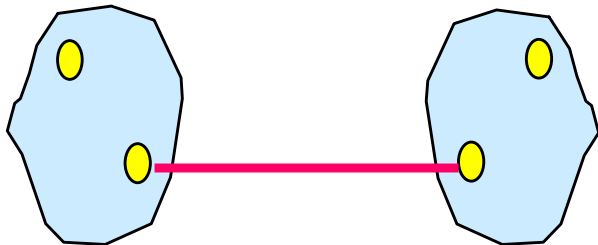
- Both Agglomerative and Divisive algorithms provide binary tree or *dendrogram*
- The final cluster is the root and each data item is a leaf
- The height of the bars indicate how close the items are



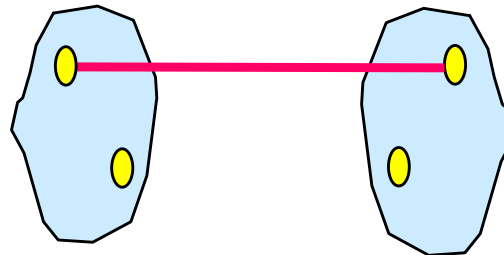
Hierarchical clustering

- What about distances between a data item and a cluster or between two clusters?
- Treat a data point as a cluster with a single item, so our only problem is to define a *linkage* method between clusters

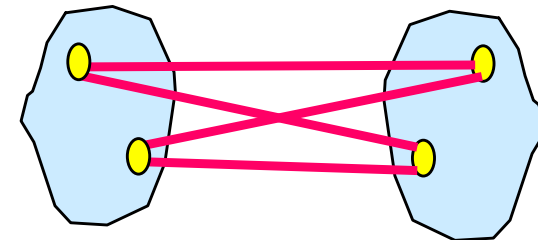
- Single linkage



- Complete linkage

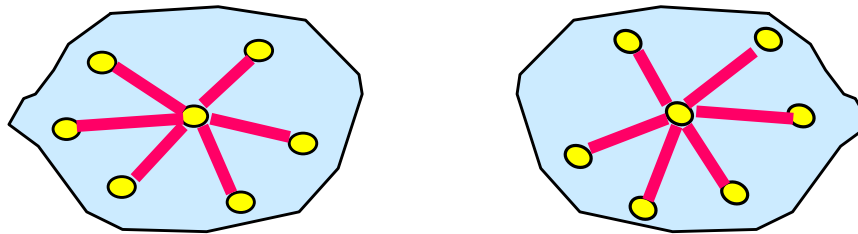


- Average linkage



Ward's method

- Variance method for hierarchical clustering which minimizes the within-cluster variance
- Less susceptible to noise and outliers
- Biased towards globular clusters
- Hierarchical analogue of k-means
- The similarity between the two clusters is the sum of squares within the clusters summed over all variables. The two clusters with the smallest increase in the overall sum of squares are combined



- **Advantages**

- Dendograms are great for visualization
- Provides hierarchical relations between clusters
- Shown to be able to capture concentric clusters

- **Disadvantages**

- Not easy to define levels for clusters
- Experiments showed that other clustering techniques outperform hierarchical clustering



Agglomerative hierarchical clustering

```
d <- dist(data.use, method ="euclidean")
```

```
hc <- hclust(d, method="ward")
```

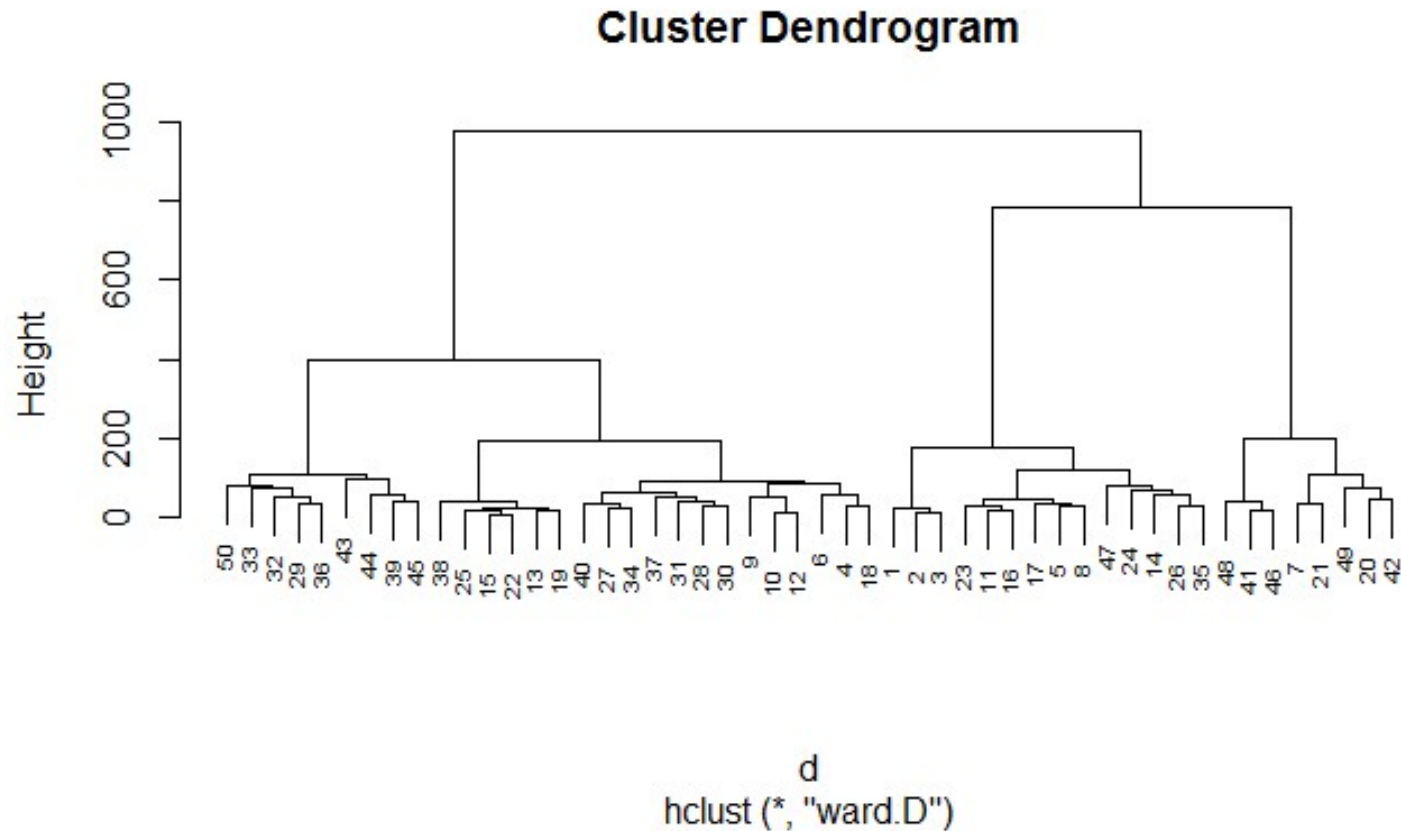
```
plot(hc)
```

- Create the distance matrix
- Perform the hierarchical clustering using various methods
- Plot the obtained dendrogram



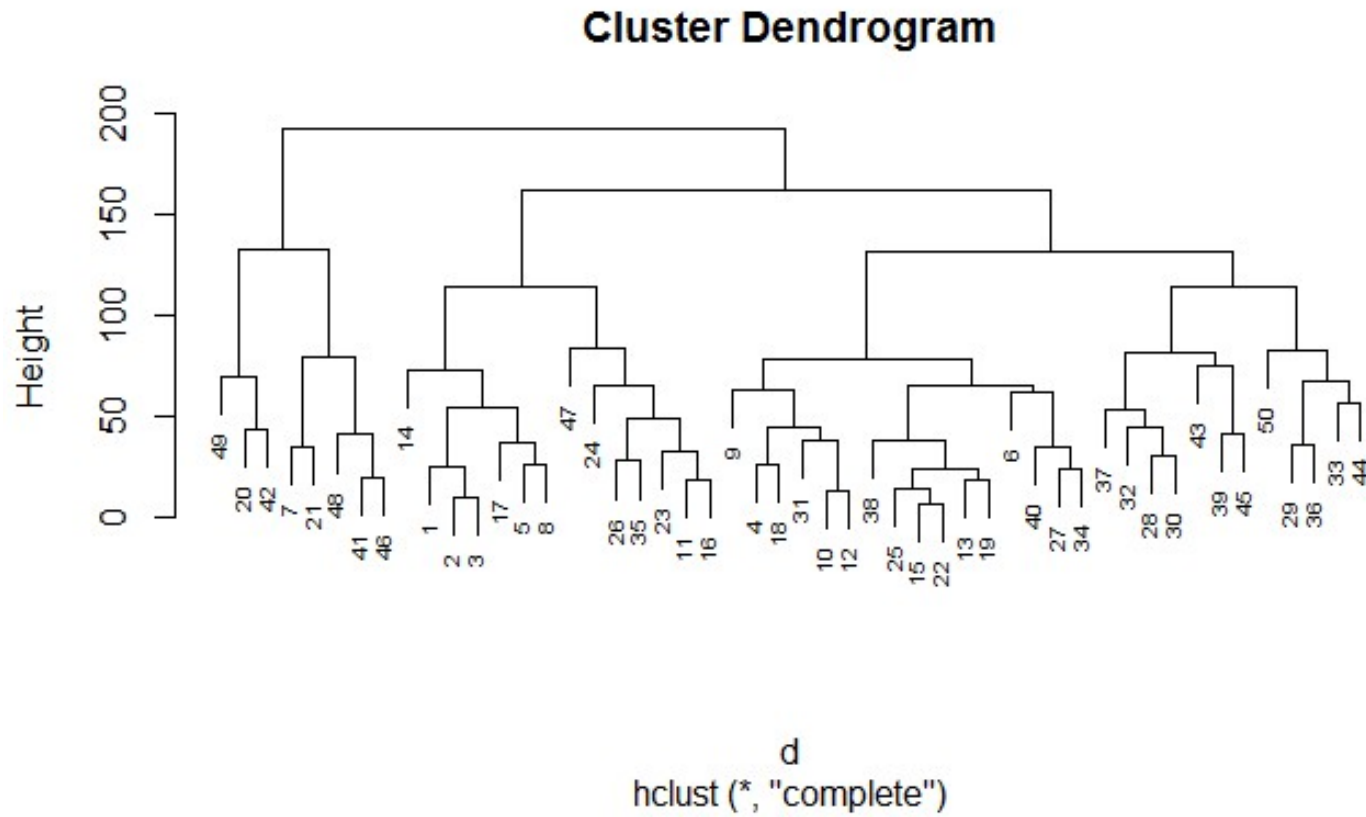
Agglomerative hierarchical clustering

```
hc <- hclust(d, method="ward")  
plot(hc)
```



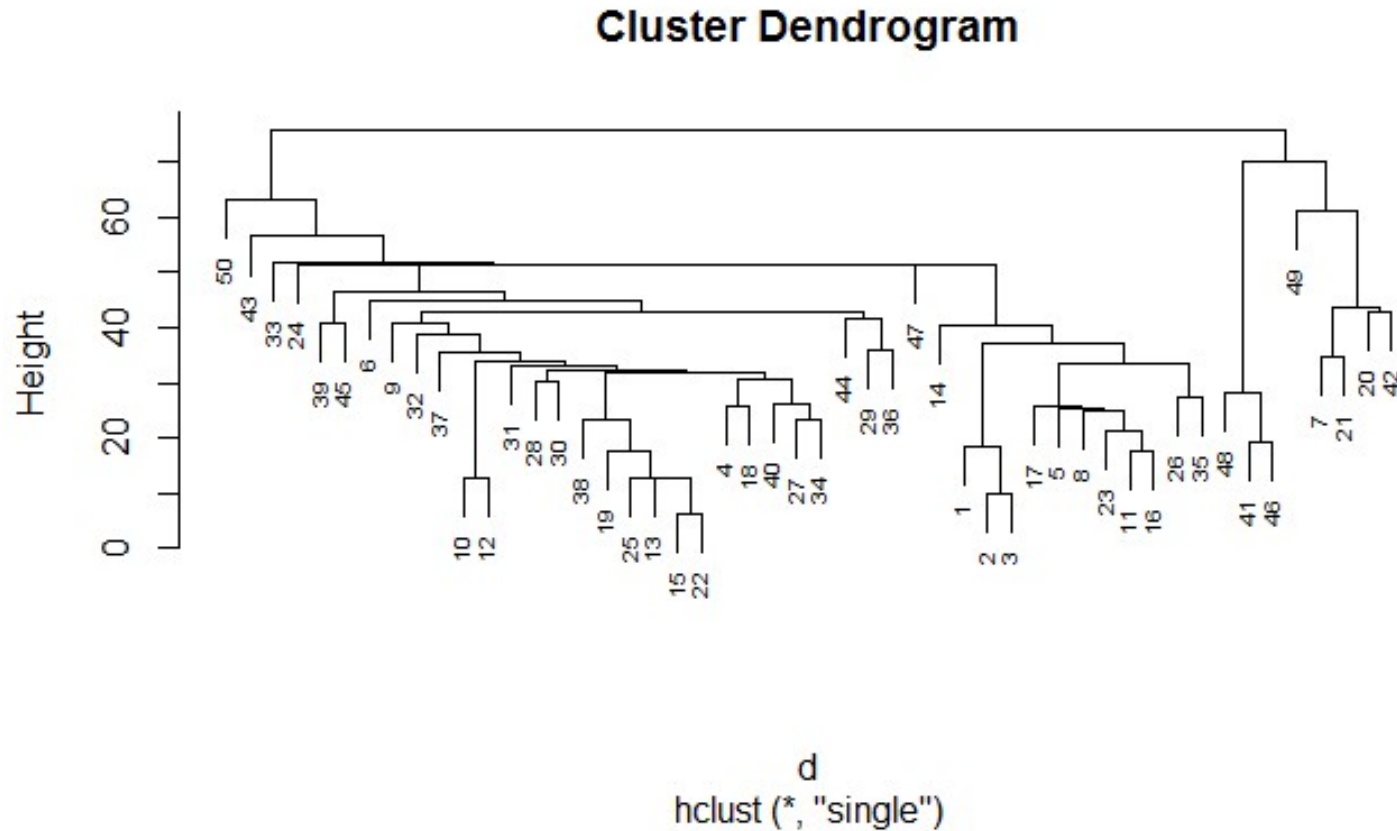
Agglomerative hierarchical clustering

```
hc <- hclust(d, method="complete")  
plot(hc)
```



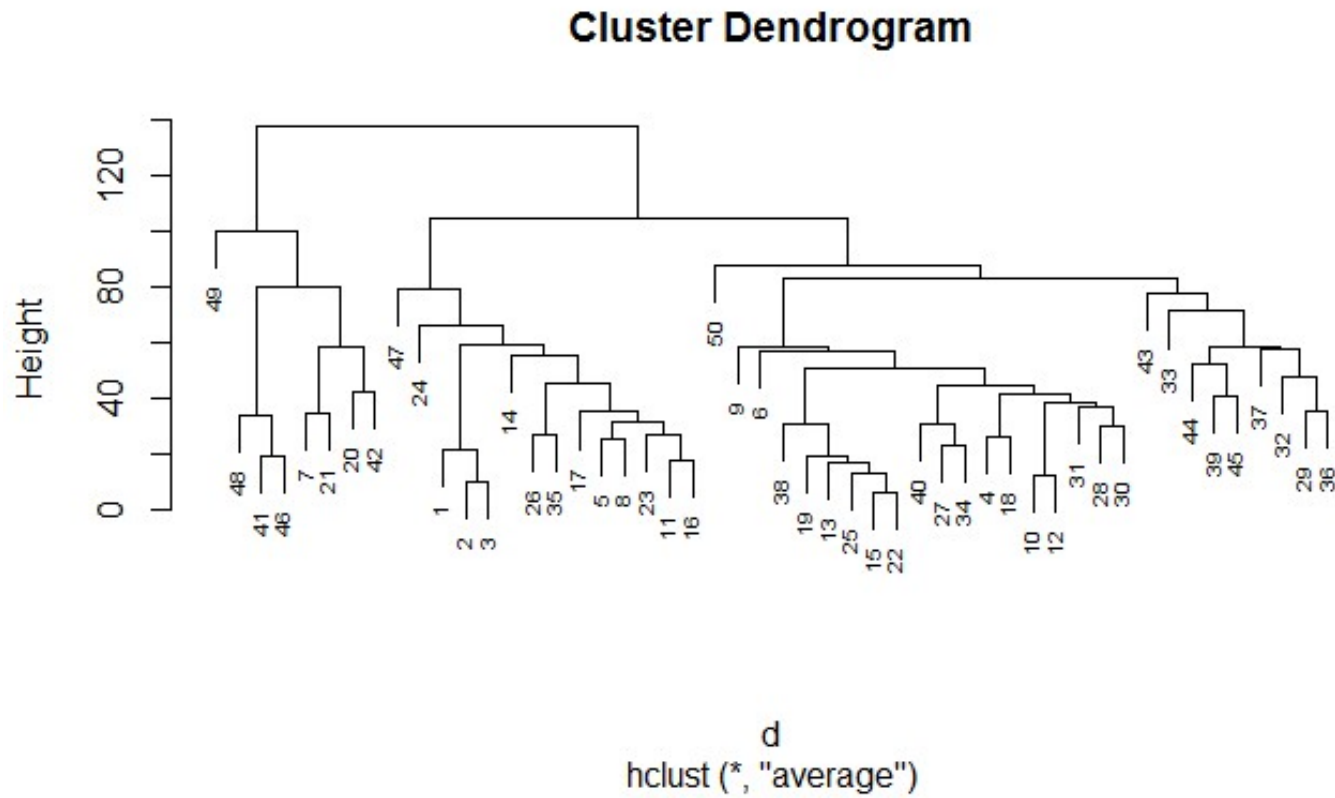
Agglomerative hierarchical clustering

```
hc <- hclust(d, method="single")  
plot(hc)
```



Agglomerative hierarchical clustering

```
hc <- hclust(d, method="average")  
plot(hc)
```



Agglomerative hierarchical clustering

```
d <- dist(data.use, method ="euclidean")
```

```
hc <- hclust(d, method="ward")
```

```
plot(hc)
```

```
clusters <- cutree(hc, k=3)
```

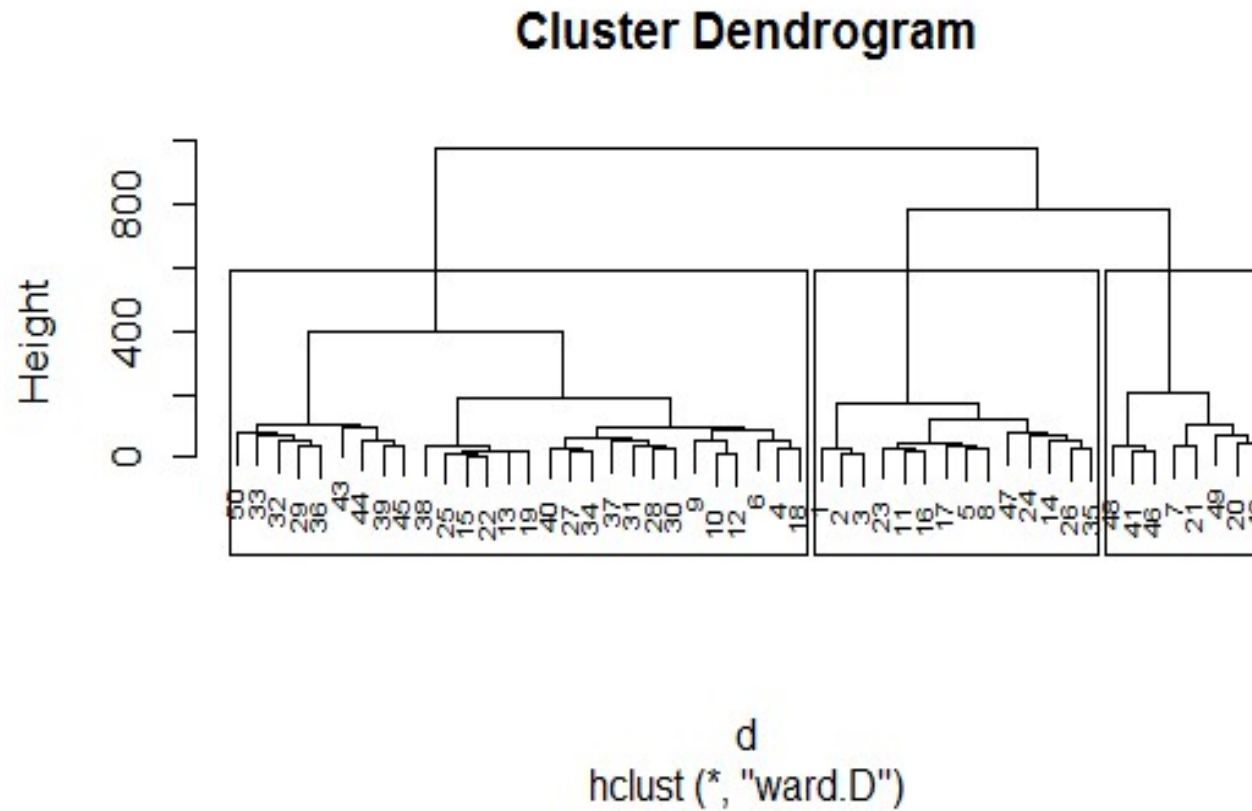
```
rect.hclust(hc, k=3, border="black")
```

- Create the distance matrix
- Perform the hierarchical clustering using various methods
- Plot the obtained dendrogram
- Cut the tree into a predefined number of clusters
- Draw borders of the predefined clusters on the dendrogram



Agglomerative hierarchical clustering

- Draw borders of the predefined clusters on the dendrogram



Additional exploration of clusters

```
table(clusters)
```

- How many entities are in each of the retained clusters?



Additional exploration of clusters

```
table(clusters)
```

- How many entities are in each of the retained clusters?

Cluster	1	2	3
Number of entities	14	28	8



Additional exploration of clusters

```
table(clusters)
```

```
aggregate(data.use,list(clusters),mean)
```

- How many entities are in each of the retained clusters?
- Gives descriptive statistics of each cluster – mean, max, min, sd



Additional exploration of clusters

```
table(clusters)
```

```
aggregate(data.use, list(clusters), mean)
```

- How many entities are in each of the retained clusters?
- Gives descriptive statistics of each cluster – mean, max, min, sd

	Academic Reputation	Employer Reputation	Faculty Student	Staff with PhD	Papers per Faculty	Citations per paper	International faculty	International Students
1	95,486	92,636	52,586	79,986	87,329	88,700	98,300	72,214
2	88,325	83,904	48,896	93,768	81,611	79,939	34,793	17,543
3	76,388	80,175	97,850	83,525	23,663	19,150	47,363	87,925



Additional exploration of clusters

```
table(clusters)
```

```
aggregate(data.use,list(clusters),mean)
```

```
dataset$University[clusters == 1]
```

- How many entities are in each of the retained clusters?
- Gives descriptive statistics of each cluster – mean, max, min, sd
- List out all entities which are in a specific cluster



Additional exploration of clusters

```
table(clusters)
```

```
aggregate(data.use, list(clusters), mean)
```

```
dataset$University[clusters == 1]
```

- How many entities are in each of the retained clusters?
- Gives descriptive statistics of each cluster – mean, max, min, sd
- List out all entities which are in a specific cluster

Universities in cluster 1

Tsinghua University Peking
University
Fudan University
Shanghai Jiao Tong University
Nanjing University Beijing Normal
University
University of Cape Town

Wuhan University
Tongji University
Sun Yat-sen University
Xi'an Jiaotong University
University of the Witwatersrand
Stellenbosch University
University of Pretoria



Divisive hierarchical clustering

```
d <- dist(data.use, method ="euclidean")
```

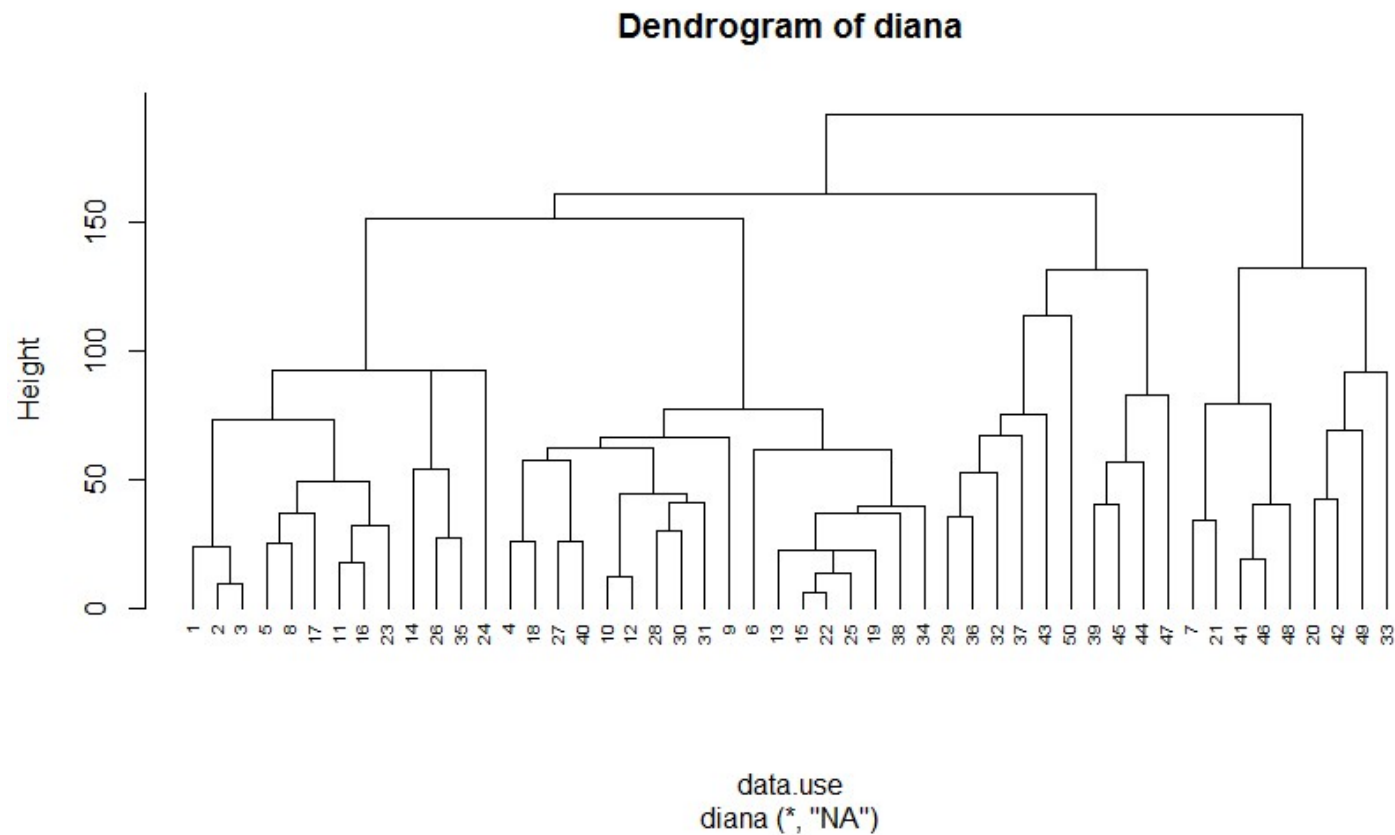
```
divisive <- diana(d)
```

```
pltree(divisive, cex = 0.6, hang = -1,  
      main = "Dendrogram of diana")
```

- Create the distance matrix
- Perform divisive hierarchical clustering on the distance matrix
- Plot the obtained dendrogram



Divisive hierarchical clustering



Divisive hierarchical clustering

```
d <- dist(data.use, method ="euclidean")
```

```
divisive <- diana(d)
```

```
pltree(divisive, cex = 0.6, hang = -1,  
       main = "Dendrogram of diana")
```

```
clustersdiv <- cutree(divisive, k=3)
```

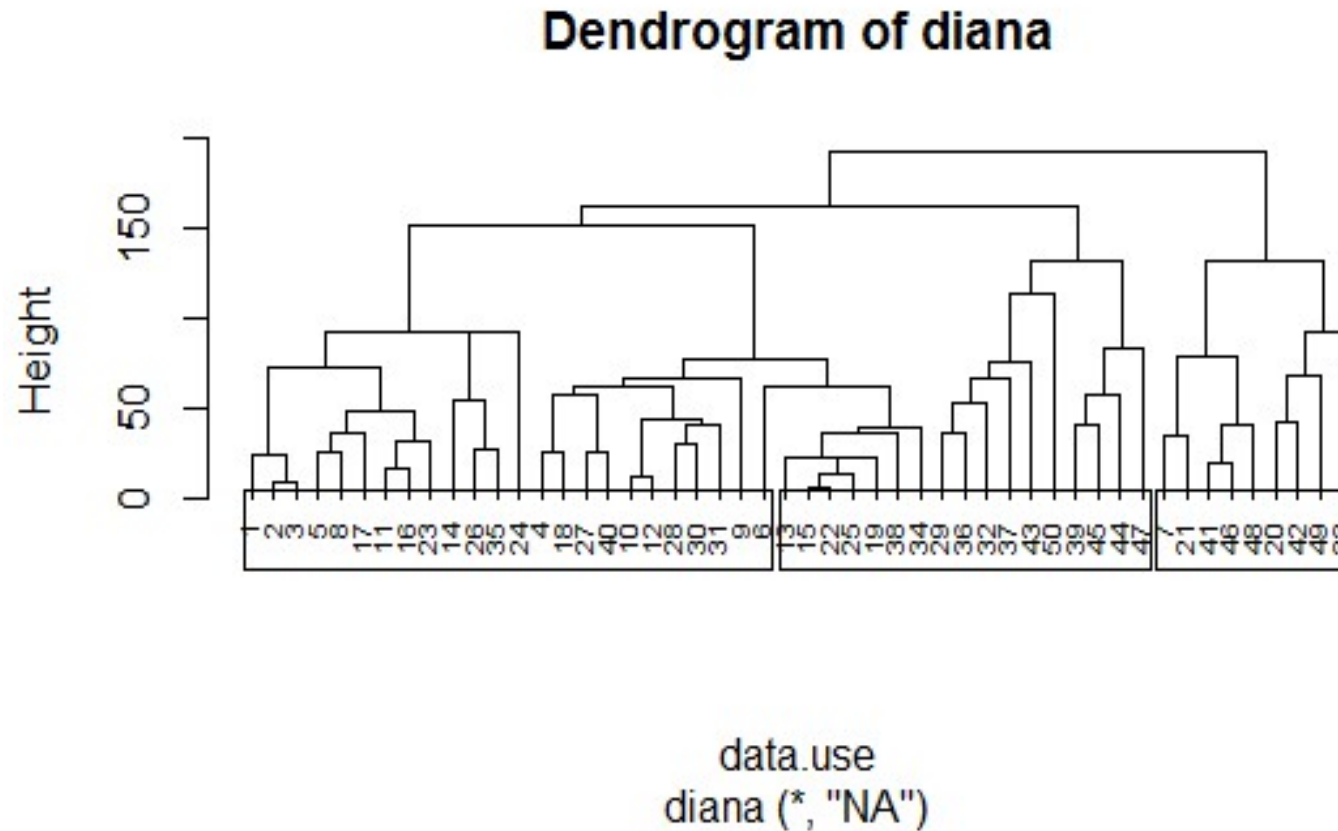
```
rect.hclust(divisive, k=3,  
            border="black")
```

- Create the distance matrix
- Perform divisive hierarchical clustering on the distance matrix
- Plot the obtained dendrogram
- Cut the tree into a predefined number of clusters
- Draw borders of the predefined clusters on the dendrogram



Divisive hierarchical clustering

- Draw borders of the predefined clusters on the dendrogram



Compare the clusters retained by AGNES and DIANA

```
data.use = subset(dataset, select=-
c(University))

d <- dist(data.use, method ="euclidean")

agglomerative <- hclust(d, method="ward")
agnesclus <- cutree(agglomerative, k=3)

divisive<-diana(d)
divisclus <- cutree(divisive, k=3)

table(agnesclus,divisclus)
```

Retain three clusters suggested
by Agnes and Diana

		Diana clusters		
		1	2	3
Agnes clusters	1	13	0	1
	2	18	1	9
	3	0	8	0



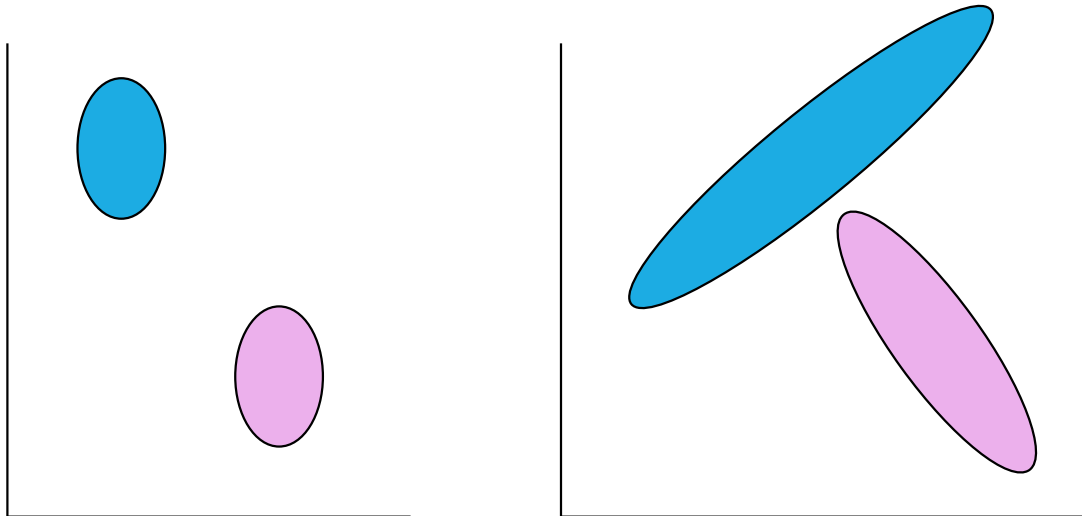
Questions that emerge

1. How to choose the right number of clusters to retain?
2. How to understand the within cluster similarity and between cluster differences in terms of distances?
3. How to display the clusters graphically?
4. How to determine cluster stability?
5. Why cluster only entities and not variables?



Distance similarity: entities in the same cluster should be closer together than entities in different clusters – **intracluster distance**

Expression similarity: entities in the same cluster should have more similar features than entities in different clusters – **intercluster distance**



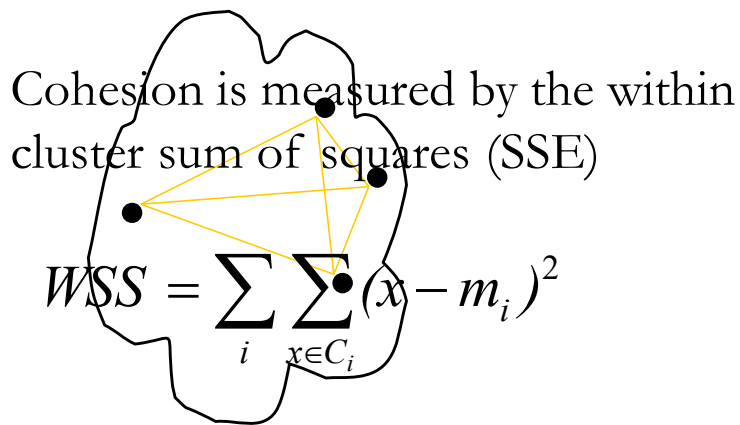
We need to be careful about how we define "closer"



Q1 & Q2

- Cluster Cohesion:**

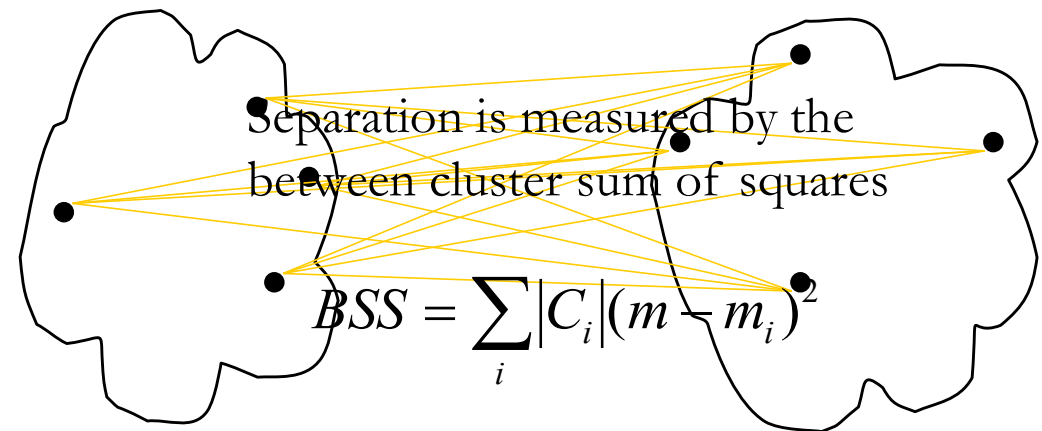
Measures how closely related are objects in a cluster



Cohesion

- Cluster Separation:**

Measure how distinct or well-separated a cluster is from other clusters



Separation



- The silhouette is a measure of how much closer points in the cluster are to one another compared to points outside the cluster.
- Observations with large and positive *sil* (~ 1) are well clustered
- Those with *sil* (~ 0) lie between clusters
- Those with negative *sil* (~ -1) are placed in the “wrong” cluster

$$\delta(i, C) = \frac{1}{\#(C)} \sum_{j \in C} d(x_i, x_j)$$

$$a_i = \delta(i, C(i)) \quad C(i) = C \text{ st } i \in C$$

$$b_i = \min_{C \neq C(i)} \delta(i, C)$$

$$sil_i = \frac{b_i - a_i}{\max\{a_i, b_i\}}$$



- To assess the clustering structure, the average silhouette width is calculated
- Based on the average value we interpret the clustering structure:

<i>Range</i>	<i>Interpretation</i>
0.71-1.0	A strong structure has been found
0.51-0.70	A reasonable structure has been found
0.26-0.50	The structure is weak and could be artificial. Try additional methods of data analysis.
<0.25	No substantial structure has been found



Silhouette plot – Example on pam() results

```
library(cluster)
sil<- silhouette(pamx$cluster,d)
plot(sil,cex.names=0.6,nmax=100,
      main="Silhouette Plot of pam() clustering")
```

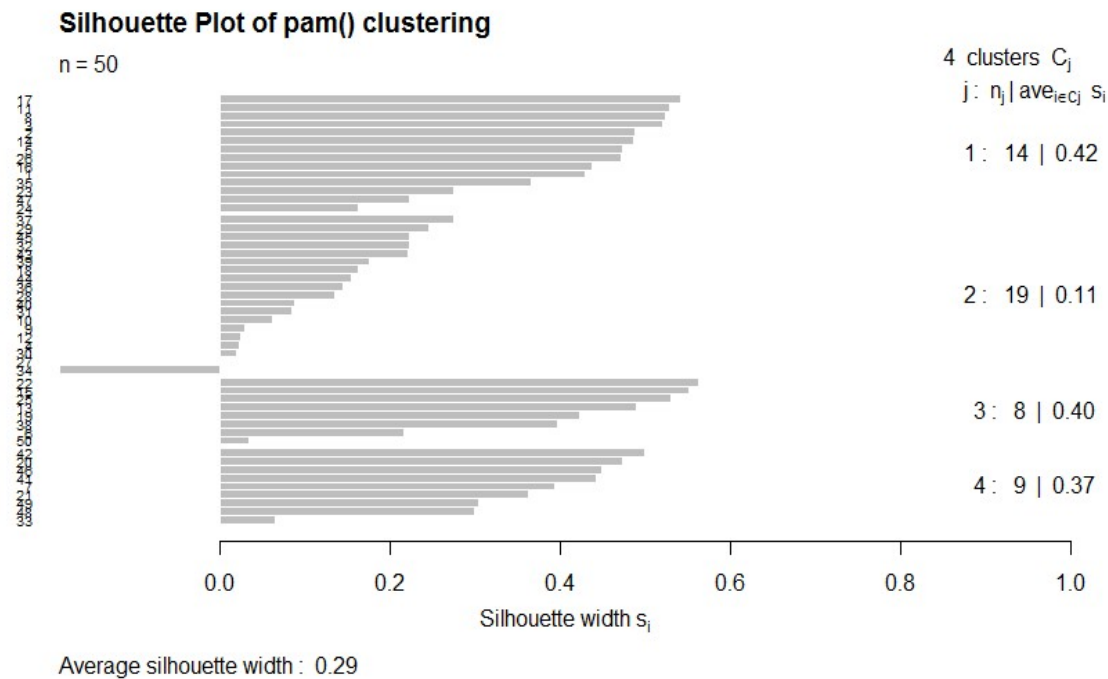
- Load the needed package
- Call the silhouette function
- Obtain the silhouette plot



Silhouette plot – Example on pam() results

```
library(cluster)
sil<- silhouette(pamx$cluster,d)
plot(sil,cex.names=0.6,nmax=100,
      main="Silhouette Plot of pam() clustering")
```

- Load the needed package
- Call the silhouette function
- Obtain the silhouette plot



Silhouette plot – Example on pamk() results

```
library(cluster)
sil<- silhouette(pamk.result$pamobject$cluster,d)
plot(sil,cex.names=0.6,nmax=100,
      main="Silhouette Plot of pamk() clustering")
```

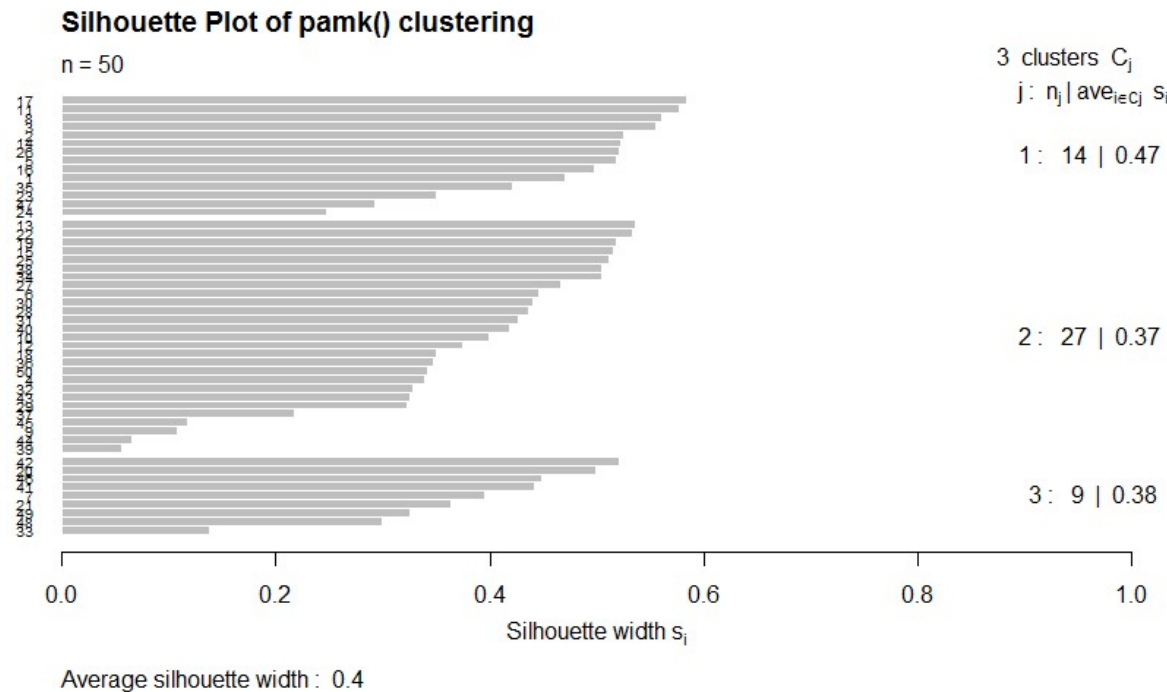
- Load the needed package
- Call the silhouette function
- Obtain the silhouette plot



Silhouette plot – Example on pamk() results

```
library(cluster)
sil<- silhouette(pamk.result$pamobject$cluster,d)
plot(sil,cex.names=0.6,nmax=100,
     main="Silhouette Plot of pamk() clustering")
```

- Load the needed package
- Call the silhouette function
- Obtain the silhouette plot



Obtain average Silhouette for different k

```
asw <- numeric(8)
for (k in 2:8)

  asw[k] <- pam(data.use, k) $ silinfo $
  avg.width

k.best <- which.max(asw)

asw
k.best
```

- Define that asw is a numeric vector?
- The interval for possible k
- Calculate and save the Average Silhouette Width
- Decide which k is the best
- Print out the obtained Average Silhouette Widths
- Print out the best number of clusters to be retained



Obtain average Silhouette for different k

```
asw <- numeric(8)
for (k in 2:8)

asw[k] <- pam(data.use, k) $ silinfo $
avg.width

k.best <- which.max(asw)

asw
k.best
```

- Define that asw is a numeric vector?
- The interval for possible k
- Calculate and save the Average Silhouette Width
- Decide which k is the best
- Print out the obtained Average Silhouette Widths
- Print out the best number of clusters to be retained

K	1	2	3	4	5	6	7	8
ASW	0.00	0.300	0.400	0.290	0.288	0.299	0.272	0.271

Three cluster structure is suggested



Visualization of asw for different k

```
plot(1:8, asw, type= "l", main = "Best number of retained clusters", xlab= "Number of clusters",  
     ylab = "Average silhouette width")
```

```
axis(1, k.best, paste("Best",k.best,sep="\n"), col = "black", col.axis = "red")
```

Function parameters of plot()

1:8 – values of k

type – type of plot – **l – line, p – points, b – both, h – histogram, s - steps**

asw – the value we want to insert on the plot

main – plot title

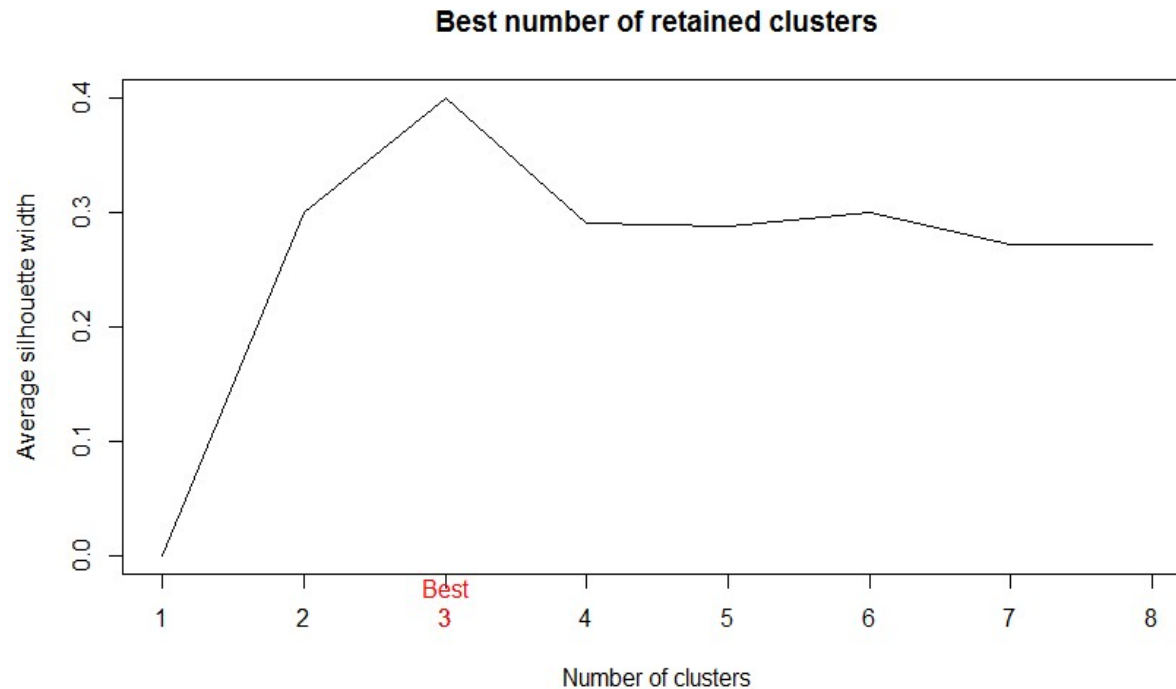
xlab and ylab – Titles of x and y axis

Function axis() - Adds an axis to the current plot, allowing the specification of the side, position, labels, and other options



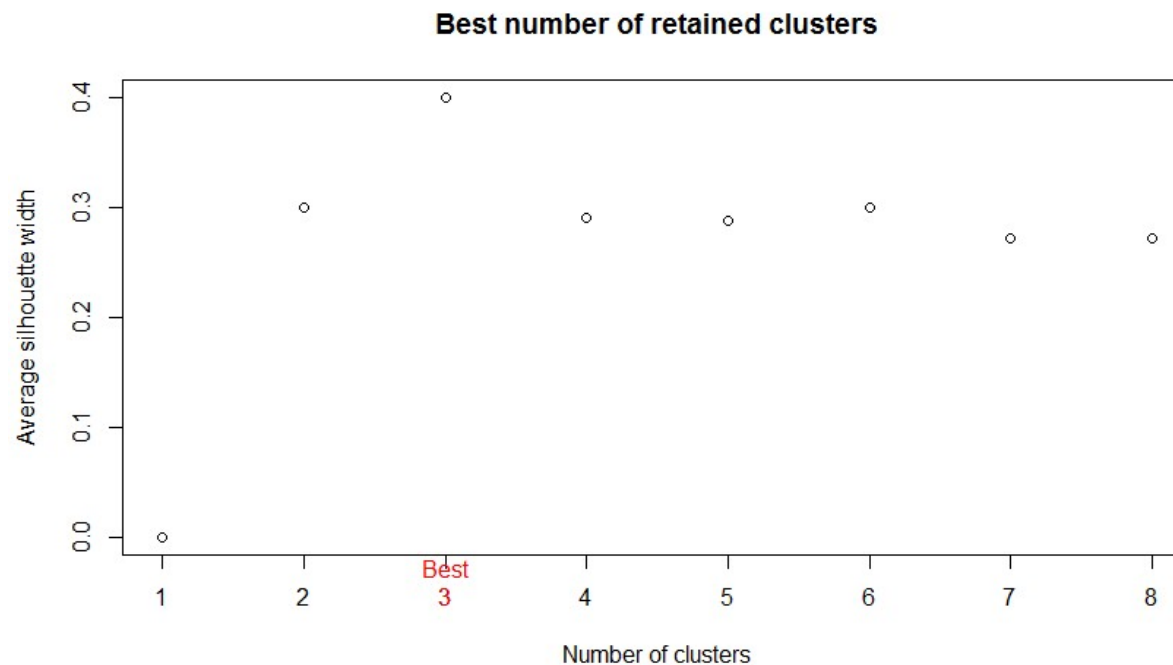
Visualization of asw for different k

```
plot(1:8, asw, type= "l", main = "Best number of retained clusters", xlab=
"Number of clusters", ylab = "Average silhouette width")
axis(1, k.best, paste("Best",k.best,sep="\n"), col = "black", col.axis = "red")
```



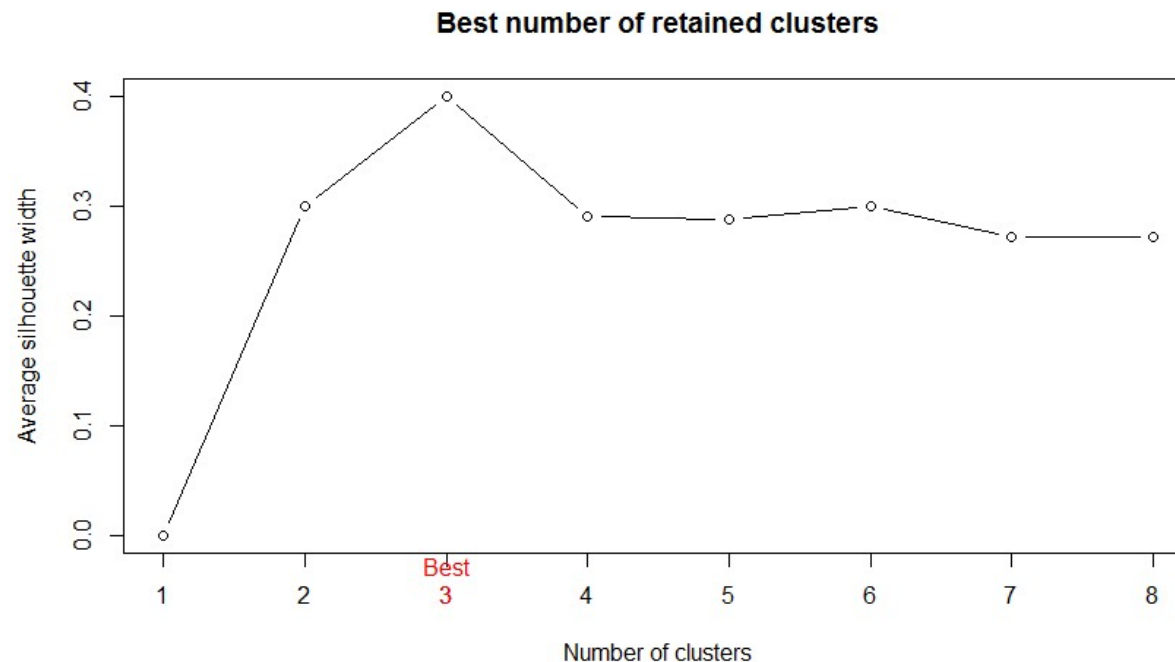
Visualization of asw for different k

```
plot(1:8, asw, type= "p", main = "Best number of retained clusters", xlab=
"Number of clusters", ylab = "Average silhouette width")
axis(1, k.best, paste("Best",k.best,sep="\n"), col = "black", col.axis = "red")
```



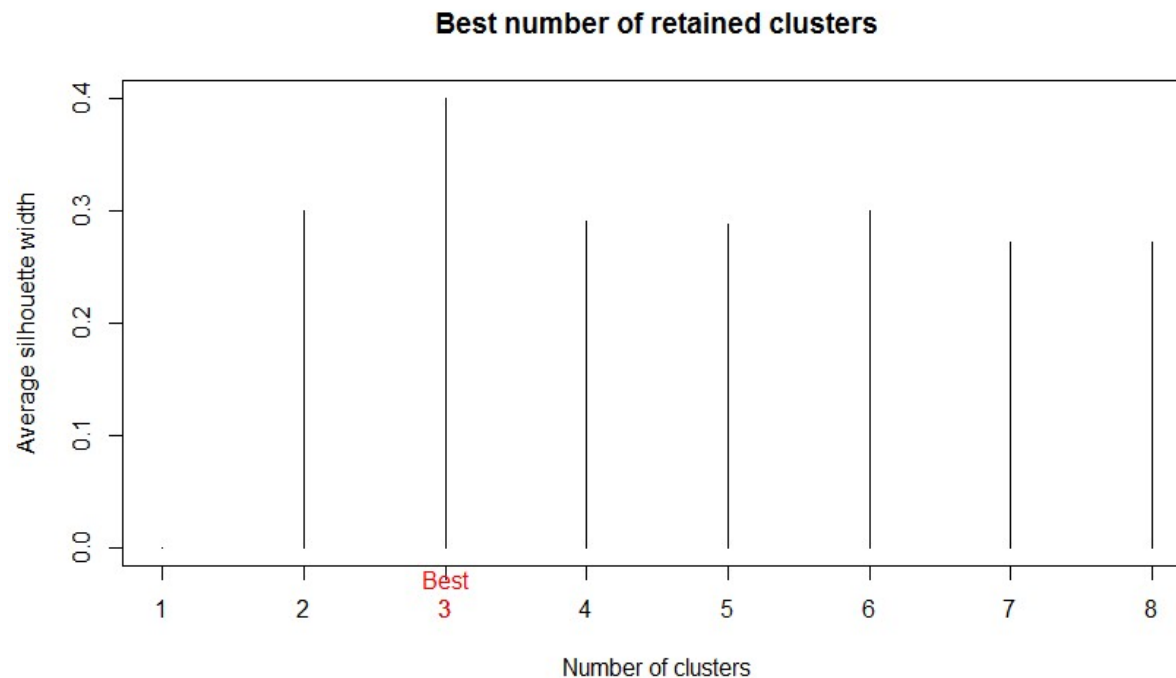
Visualization of asw for different k

```
plot(1:8, asw, type= "b", main = "Best number of retained clusters", xlab=
"Number of clusters", ylab = "Average silhouette width")
axis(1, k.best, paste("Best",k.best,sep="\n"), col = "black", col.axis = "red")
```



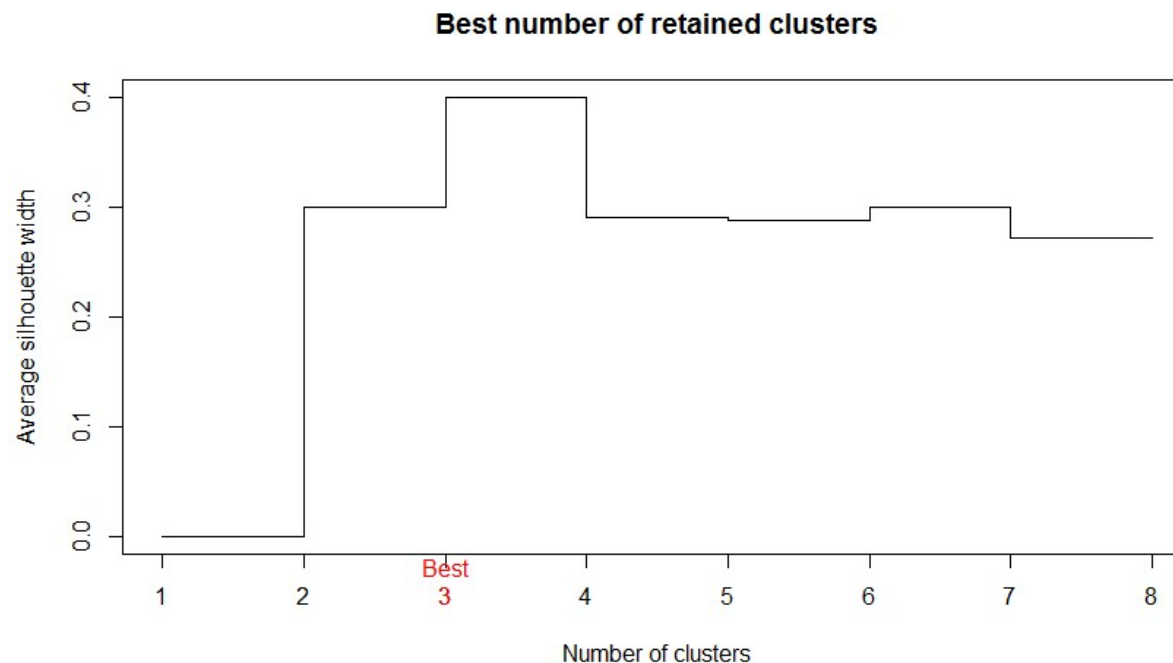
Visualization of asw for different k

```
plot(1:8, asw, type= "h", main = "Best number of retained clusters", xlab=
"Number of clusters", ylab = "Average silhouette width")
axis(1, k.best, paste("Best",k.best,sep="\n"), col = "black", col.axis = "red")
```



Visualization of asw for different k

```
plot(1:8, asw, type= "s", main = "Best number of retained clusters", xlab=
"Number of clusters", ylab = "Average silhouette width")
axis(1, k.best, paste("Best",k.best,sep="\n"), col = "black", col.axis = "red")
```



K-means – Total within sum of squares

To determine the true number of clusters when employing k-means algorithms the Total within sum of squares is used

A convenient plot is the one which gives information on the Total within SS for different number of retained clusters

```
range<-2:10
tries <-100
avg.totw.ss <-integer(length(range))
for(v in range){ v.totw.ss <-
integer(tries)
for(i in 1:tries){
  k.temp <-kmeans(data.use,centers=v)
  v.totw.ss[i] <-k.temp$tot.withinss}
avg.totw.ss[v-1] <-mean(v.totw.ss)}
```

- Range of number of retained clusters
- Number of times to run k-means
- Create a vector to store the avg.totw.ss
- For each v in range and for each try calculate and store the Total within SS
- Store the Total within SS
- Average the Total within SS



K-means – Total within sum of squares

Plot the obtained Total within SS for different k

A sharp bend in the graph can suggest the appropriate number of clusters

```
plot(range, avg.totw.ss, type="b", main="Total within SS for various number of  
retained clusters", ylab="Average Total within Sum of Squares", xlab="Number of  
retained clusters")
```

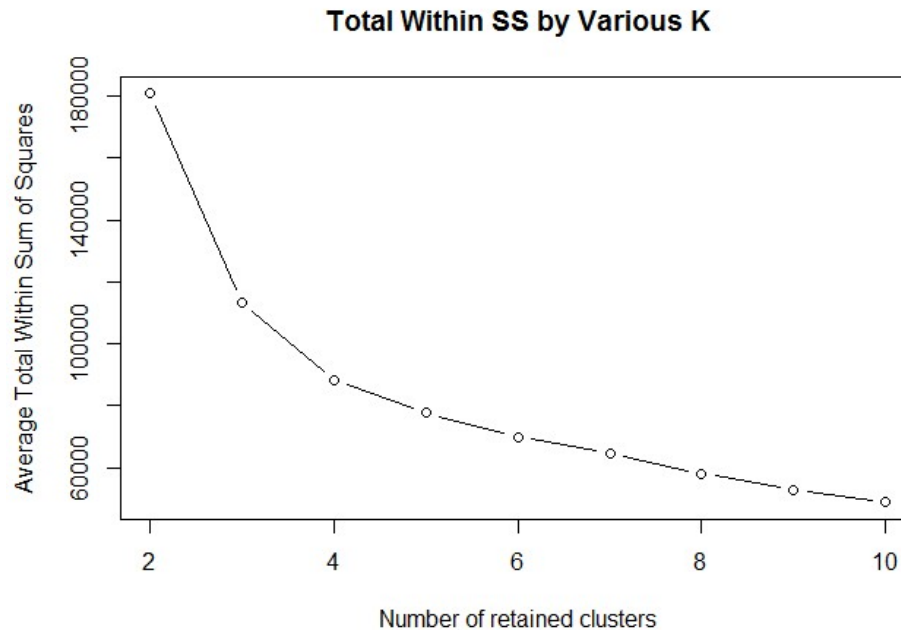


K-means – Total within sum of squares

Plot the obtained Total within SS for different k

A sharp bend in the graph can suggest the appropriate number of clusters

```
plot(range, avg.totw.ss, type="b", main="Total within SS for various number of  
retained clusters", ylab="Average Total Within Sum of Squares", xlab="Number of  
retained clusters")
```



- Example of intracluster measures besides silhouette:
 - Calinski and Harabasz (CH) index
 - Duda index
 - Pseudot² index
 - Cubic Clustering Criterion (CCC)
 - Hartigan index
 - And many more
- Should we calculate each of them separately for different values of k?



Package NbClust

- NbClust package provides **30 indices** for determining the number of clusters and proposes to use the **best clustering scheme** from the different results obtained **by varying** all combinations of number of clusters, distance measures, and clustering methods
- The best cluster structure is chosen according to the majority rule
- More information on the package and indices it calculates:
<https://cran.r-project.org/web/packages/NbClust/NbClust.pdf>



Application of NbClust()

```
install.packages("NbClust")  
library(NbClust)
```

- Install the NbClust package
- Load the NbClust package

```
NbClust(data = X, diss = NULL, distance = "distance", min.nc = 2, max.nc = 15,  
        method = "method", index = "all", alphaBeale = 0.1)
```

Function parameters of NbClust()

data – data set on which we want to perform clustering

diss – dissimilarity matrix – If we enter data set in data, then diss should be NULL

distance – type of distance used to obtain the dissimilarity matrix

min.nc & max.nc – minimum and maximum number of clusters we take into consideration

method – clustering method - "ward.D2", "single", "complete", "average", "centroid", "kmeans "

index – which indices we want to calculate



Application of NbClust()

```
install.packages("NbClust")
```

```
library(NbClust)
```

```
indices<-NbClust(data = data.use, diss = NULL, distance = "euclidean",  
min.nc = 2, max.nc = 15, method = "kmeans", index = "all", alphaBeale = 0.1)
```



Application of NbClust()

```
install.packages("NbClust")
```

```
library(NbClust)
```

```
indices<-NbClust(data = data.use, diss = NULL, distance = "euclidean",  
min.nc = 2, max.nc = 15, method = "kmeans", index = "all", alphaBeale = 0.1)
```

Among all indices	Conclusion
1 proposed 2 as the best number of clusters	According to the majority rule, the best number of clusters is 3
14 proposed 3 as the best number of clusters	
2 proposed 4 as the best number of clusters	
1 proposed 6 as the best number of clusters	
2 proposed 7 as the best number of clusters	
1 proposed 14 as the best number of clusters	
2 proposed 15 as the best number of clusters	



Application of NbClust()

`indices$Best.nc`

`indices$All.index`

`indices$Best.partition`

- Tells how many clusters each index suggests and the critical value of each index
- Gives all index values for all considered number of clusters
- Give the best partition of entities to the best number of clusters



Application of NbClust()

`indices$Best.nc`

`indices$All.index`

`indices$Best.partition`

- Tells how many clusters each index suggests and the critical value of each index
- Gives all index values for all considered number of clusters
- Give the best partition of entities to the best number of clusters

Entity	1	2	3	4	...	47	48	49	50
Cluster	1	1	1	2	...	1	3	3	2



Application of NbClust()

```
indices$Best.nc
```

```
indices$All.index
```

```
indices$Best.partition
```

```
dataset$University[indices$Best.partition == 1]
```

- Tells how many clusters each index suggests and the critical value of each index
- Gives all index values for all considered number of clusters
- Give the best partition of entities to the best number of clusters
- Lists all universities within cluster



Q3 – Display the clusters graphically

- So far we introduced several functions to graph clusters
 - `clusplot()` – plot `kmeans()` clustering
 - `plot()` – plots both cluster plot and silhouette plot
 - `rect.hclust()` – cut the dendrogram
- Let's see more advanced plots



```
plot(dataset$AcademicReputation, dataset$EmployerReputation, type="n",  
      xlab="Academic Reputation", ylab="Employer Reputation", main="Values of  
      Academic reputation and Employer reputation per pamk() retained  
      cluster")  
  
text(x=dataset$AcademicReputation, y=dataset$EmployerReputation,  
     col=pamk.result$pamobject$cluster+1)
```

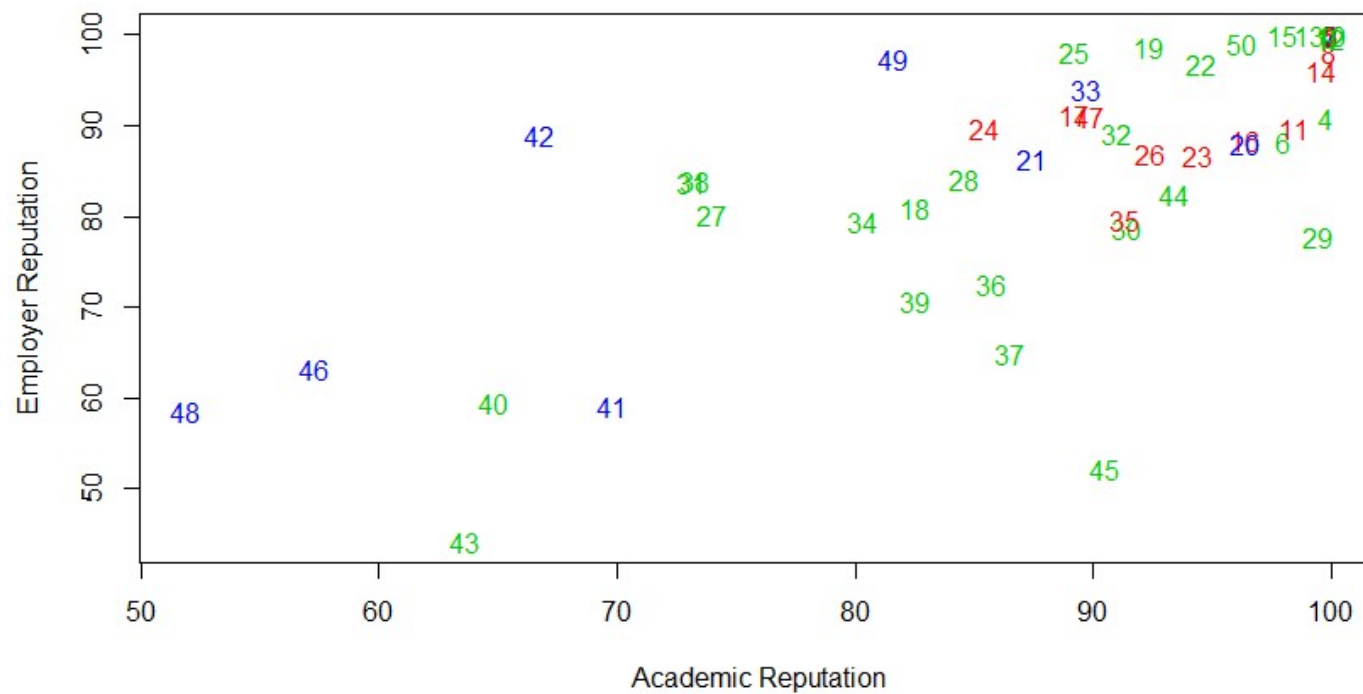
Function `plot()` – here choose two variables whose values per cluster you are interested in plotting and give axis and main title

Function `text()` – again choose the two same variables and group them by clusters

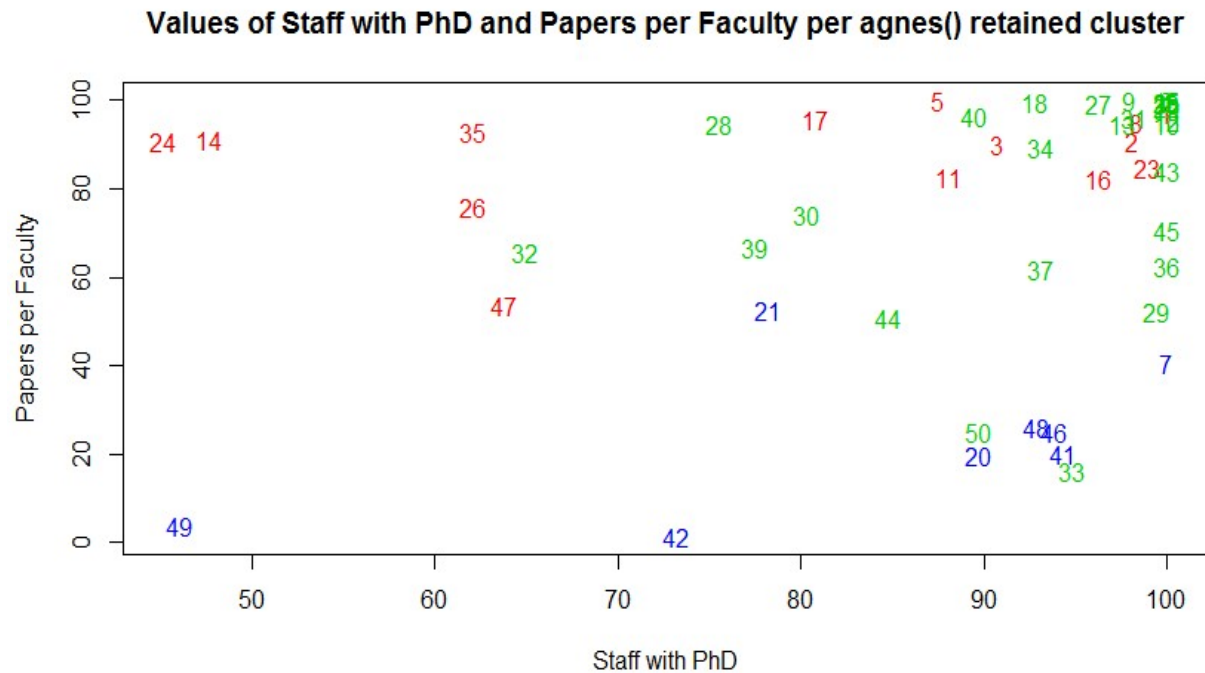
The chosen example takes variables Academic Reputation and Employer Reputation and groups them according to the `pamk()` results



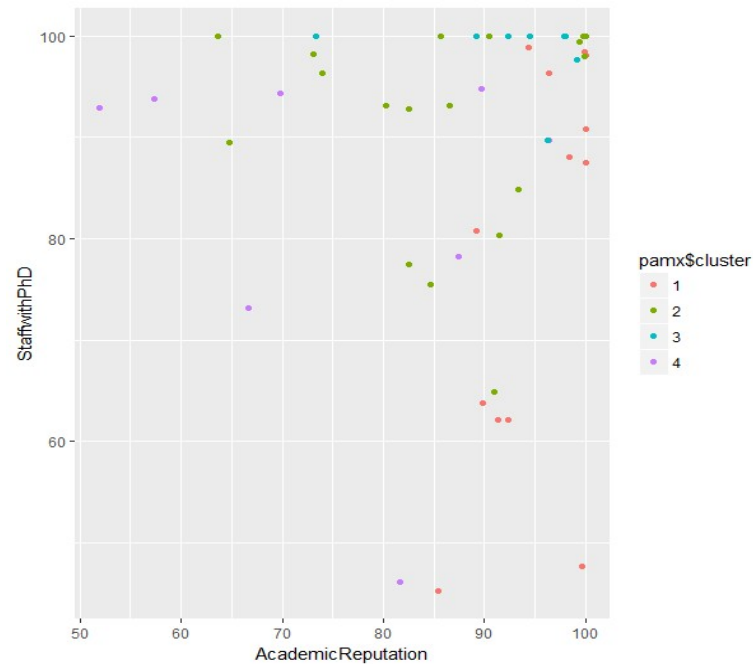
Values of Academic reputation and Employer reputation per pamk() retained cluster



```
plot(dataset$StaffwithPhD, dataset$PapersperFaculty, type="n",
      xlab="Staff with PhD", ylab="Papers per Faculty", main="Values of Staff
      with PhD and Papers per Faculty per agnes() retained cluster")
text(x=dataset$StaffwithPhD, y=dataset$PapersperFaculty, col=agnesclus+1)
```



```
install.packages("ggplot2")  
library(ggplot2)  
pamcluster$cluster <- as.factor(pamcluster$cluster)  
graph<-ggplot(dataset, aes(AcademicReputation, StaffwithPhD,  
color = pamcluster$cluster)) + geom_point()  
plot(graph)
```

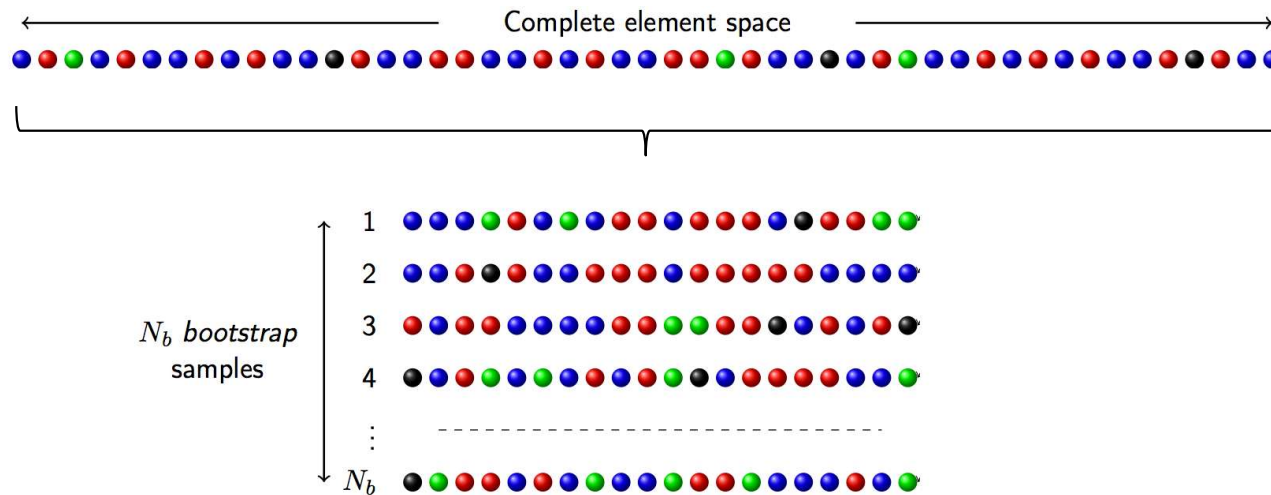


```
graph + labs(title= "Academic Reputation and Staff with PhD per cluster",
x="Academic Reputation", y="Staff with PhD", color="Cluster")
```



Q4 - How to determine cluster stability?

- A good approach to inspect the cluster stability is to use resampling
- In other words, resample your data (via bootstrap or by adding small noise to it) and compute the "closeness" of the resulting partitions, as measured by Jaccard similarities.



Q4 - How to determine cluster stability?

Jaccard similarities

- Jaccard Index = (the number in both sets) / (the number in either set) * 100
- The formula notation is:
$$J(x, y) = \frac{|x \cap y|}{|x \cup y|}$$
- Interpretation of Jaccard Index values:
 - Two sets that share all members would be 100% similar
 - The midway point, 50% , means that the two sets share half of the members
 - If they share no members, they are 0% similar.



clusterboot()

```
library(fpc)
```

```
km.boot <- clusterboot(data.use, B=20, bootmethod="boot",  
                        clustermethod=kmeansCBI, krange=3, seed=15555)
```

Function parameters clusterboot() parameters:

B - Number of resampling runs

bootmethod - bootstrap method

clustermethod - clustering method employed – kmeansCBI, hclustCBI,
claraCBI, pamkCBI,...

krange - numbers of clusters to be compared



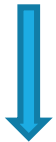
clusterboot()

```
library(fpc)
```

```
km.boot <- clusterboot(data.use, B=20, bootmethod="boot",  
clustermethod=kmeansCBI, krange=3, seed=15555)
```

```
km.boot$result
```

- Print out the results



Gives information on cluster sizes, cluster means, clustering vector, and within cluster sum of squares by cluster



clusterboot()

```
km.boot$bootresult
```

- Prints out the values of the Jaccard index



clusterboot()

```
km.boot$bootresult
```

- Prints out the values of the Jaccard index

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]	[,10]	[,11]	[,12]	[,13]	[,14]	[,15]	[,16]	[,17]	[,18]	[,19]	[,20]
[1,]	1	1	1	0.8888889	0.8333333	1	1	0.7000000	1	1	0.8500000	0.8823529	1	1	0.7272727	0.8421053	1	1	1	0.3333333
[2,]	1	1	1	1.0000000	1.0000000	1	1	0.5000000	1	1	1.0000000	1.0000000	1	1	0.6250000	0.5000000	1	1	1	0.8571429
[3,]	1	1	1	0.7500000	0.7272727	1	1	0.9090909	1	1	0.6666667	0.8181818	1	1	0.2727273	1.0000000	1	1	1	0.5500000



clusterboot()

```
km.boot$bootresult
```

```
plot(km.boot)
```

- Prints out the values of the Jaccard index
- Plots the histogram of Jaccard index per cluster

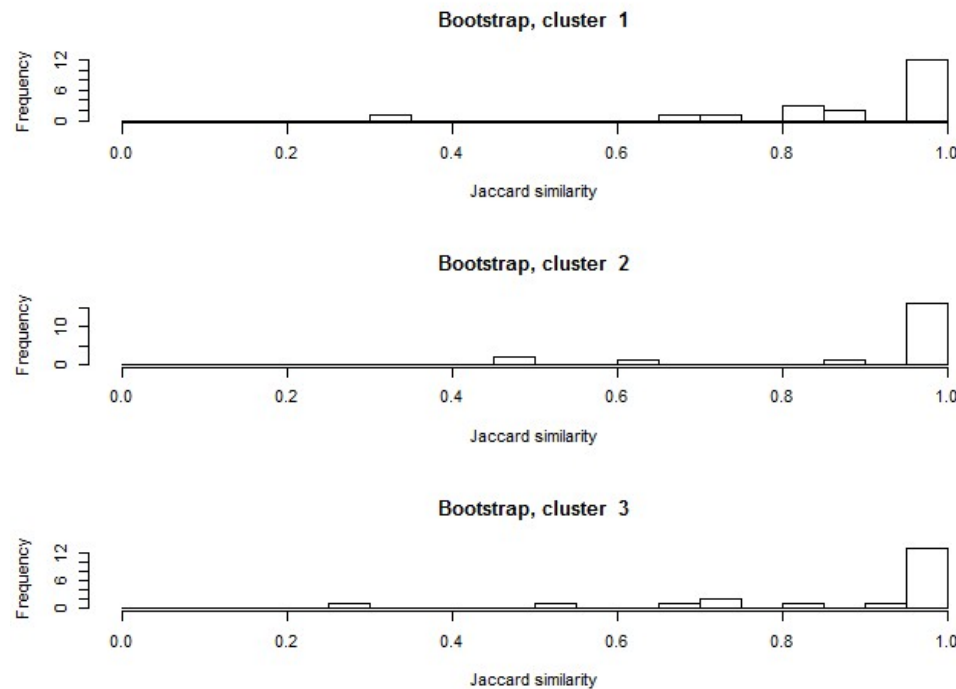


clusterboot()

```
km.boot$bootresult
```

```
plot(km.boot)
```

- Prints out the values of the Jaccard index
- Plots the histogram of Jaccard index per cluster



clusterboot() - plot

```
library(fpc)
```

```
km.boot <- clusterboot(data.use, B=20, bootmethod="boot",  
                        clustermethod=kmeansCBI, showplots=TRUE, krange=3, seed=15555)
```

showplots=TRUE – A plot of the first two dimensions of the resampled data set is shown for every resampling run. The last plot shows the original data set.



Animation can be made from the obtained plots

Animations can be easily made online for free



clusterboot() - plot

